Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Measurement and Information Systems

Analysis of Multilingual Automatic Speech Recognition Approaches

Bachelor's Thesis

Author
Munkhchimeg Sergelen

Advisor
dr. Péter Mihajlik

January 15, 2026

Contents

Kivonat **7**

Abstract **8**

1 Introduction **9**

1.1 Motivation and Background 9

1.2 Problem Statement 10

1.3 Research Questions 10

1.4 Contributions 11

1.5 Key Findings Preview 11

1.6 Thesis Organization 12

2 Background and Related Work **13**

2.1 Automatic Speech Recognition Fundamentals 13

2.1.1 Task Definition 13

2.1.2 Neural ASR Architectures 13

2.1.3 Evaluation Metrics 14

2.2 Multilingual ASR 14

2.2.1 Motivation for Multilingual Systems 14

2.2.2 Technical Challenges 14

2.2.3 Multilingual Modeling Approaches 15

2.3 Language Identification 15

2.3.1 Task Definition and Role in ASR 15

2.3.2 LID Approaches 16

2.3.3 LID Error Propagation 16

2.4 Related Work 16

2.4.1 OpenAI Whisper 16

2.4.2 OmniLingual and CTC-Based Models 17

2.4.3 Language-Specific ASR Development: Chimege 17

2.4.4 Research Gaps . . . 17

3 Methodology **19**

3.1 Evaluation Framework . . . 19

3.2 Languages and Dataset . . . 19

3.2.1 Language Selection . . . 19

3.2.2 Dataset Selection and Preparation . . . 20

3.2.3 Dataset Statistics . . . 20

3.3 ASR Systems . . . 21

3.4 Inference Modes . . . 22

3.4.1 Mode A: LID→ASR Pipeline . . . 22

3.4.2 Mode B: Language-Hinted ASR . . . 22

3.4.3 Mode Comparison Methodology . . . 22

3.5 Evaluation Metrics . . . 23

3.5.1 Word Error Rate (WER) . . . 23

3.5.2 Character Error Rate (CER) . . . 23

3.5.3 Real-Time Factor (RTF) . . . 23

3.5.4 Language Identification Accuracy . . . 24

3.5.5 Resource Usage Metrics . . . 24

3.6 Experimental Setup and Reproducibility . . . 24

3.6.1 Hardware Configuration . . . 24

3.6.2 Software Environment . . . 25

3.6.3 Execution Workflow . . . 25

3.6.4 Reproducibility Measures . . . 25

4 Results **27**

4.1 Recognition Quality . . . 27

4.1.1 Word Error Rate (WER) by Model and Language . . . 27

4.1.2 Character Error Rate (CER) by Model and Language . . . 28

4.1.3 Error Distribution Analysis . . . 29

4.1.4 Error Variability and Metric Analysis . . . 30

4.1.5 WER-CER Relationship . . . 31

4.1.6 Performance Overview . . . 31

4.2 Processing Efficiency Analysis . . . 31

4.2.1 Real-Time Factor (RTF) by Model and Language . . . 31

4.2.2 Speed-Accuracy Trade-off Analysis . . . 35

4.2.3 CPU vs GPU Performance . . . 35

4.3 Language Identification Accuracy . . . 35
4.3.1 LID Accuracy by Language . . . 36
4.3.2 LID Confidence Distribution . . . 37
4.3.3 LID Confusion Patterns . . . 37
4.4 Resource Consumption Analysis . . . 37
4.4.1 CPU and Memory Usage . . . 37
4.5 Audio Duration Analysis . . . 39
4.5.1 Duration Distribution . . . 39
4.5.2 Duration Histogram . . . 39
4.5.3 Performance by Duration Category . . . 39
4.6 Chapter Summary . . . 40

5 Discussion **42**
5.1 Interpretation of Key Findings . . . 42
5.1.1 The Mongolian Catastrophe: Language Inequality in Multilingual Models . . . 42
5.1.2 CTC Architecture with Consistently High Speed . . . 43
5.1.3 Speed-Accuracy Trade-offs: No Universal Solution . . . 43
5.2 Failure Mode Analysis . . . 43
5.2.1 LID Confusion and Error Cascading . . . 43
5.2.2 Low-Resource Language Degradation . . . 43
5.2.3 Speed Variability and Predictability . . . 44
5.3 Practical Deployment Recommendations . . . 44
5.3.1 Model Selection Guidelines . . . 44
5.3.2 Deployment Architecture Recommendations . . . 44
5.3.3 Hardware and Scaling . . . 45
5.4 Limitations . . . 45
5.4.1 Dataset Limitations . . . 45
5.4.2 Experimental Limitations . . . 46
5.4.3 Metric Limitations . . . 46
5.5 Comparison with Prior Work . . . 46
5.5.1 Consistent with Prior Findings . . . 46
5.5.2 Novel Contributions . . . 46

6 Conclusions and Future Work **47**
6.1 Answers to Research Questions . . . 47
6.2 Limitations and Lessons Learned . . . 47

6.3 Future Work 48
6.3.1 Extended Evaluation and Model Optimization 48
6.3.2 Sign Language Accessibility 48
6.4 Concluding Remarks 48

References **49**

STUDENT DECLARATION

I, the undersigned *Sergelen Munkhchimeg*, hereby declare that I prepared this thesis independently, without unauthorized assistance, and only using the indicated sources (literature, tools, etc.). All parts taken from other sources, either verbatim or paraphrased with the same meaning, are clearly indicated with citation of the source.

I consent to the publication of the basic data of this work (author(s), title, abstract in English and Hungarian, year of completion, supervisor(s) name) by BME VIK in publicly accessible electronic form, and the full text of the work through the university's internal network (or for authenticated users). I declare that the submitted work and its electronic version are identical. In the case of confidential theses with dean's permission, the text of the thesis will only become accessible after 3 years.

Budapest, January 15, 2026

Sergelen Munkhchimeg
student

Kivonat

Ez a diplomamunka a többnyelvű automatikus beszédfelismerést (ASR) értékeli két megközelítés összehasonlításával: (i) automatikus nyelvfelismerés, majd átírás (LID→ASR), és (ii) nyelv-megadásos ASR, ahol a célnyelv explicit módon meg van adva. Négy nyelvet értékelünk (spanyol, francia, magyar, mongol) a Mozilla Common Voice v23.0 adatbázisból, hat modell felhasználásával: három Whisper modell (small, medium, large-v3), amelyek az enkóder-dekóder architektúrát képviselik, és három OmniLingual modell (CTC 300M, CTC 1B, LLM 1B), amelyek a CTC-alapú megközelítéseket képviselik. A minőséget szóhibaarány (WER), karakterhibaarány (CER) és nyelvfelismerési pontosság alapján mérjük, míg a hatékonyságot valós idejű tényező (RTF) és erőforrás-fogyasztás alapján értékeljük. Az eredmények azt mutatják, hogy a Whisper-small nehézségekkel küzd a mongol nyelvnél (40×-szer lassabb, mint a spanyolnál GPU-n, RTF 4,37 vs 0,11), míg a nagyobb Whisper modellek (medium, large-v3) valós idejű mongol feldolgozást érnek el (RTF 0,55). Az OmniLingual CTC modellek konzisztens sebességet tartanak fenn minden nyelvnél (RTF 0,08–0,10), a mongolt 44–49×-szer gyorsabban dolgozzák fel, mint a Whisper-small. Az OmniLingual LLM 1B éri el a legjobb mongol pontosságot (WER 45,46%) versenyképes sebességgel. A LID→ASR 93,25%-os összesített pontosságot ér el, de a mongol 80%-ra csökken, ami hibákhoz vezet az átírási szakaszban; a nyelv-megadásos mód ajánlott, ha a nyelv ismert. A diplomamunka telepítési ajánlásokat ad a hardver-modell interakciók alapján, és jövőbeli kutatási irányokat javasol a kódváltás, streaming ASR és alacsony erőforrású nyelvek robusztussága terén.

Abstract

This thesis evaluates multilingual automatic speech recognition (ASR) by comparing two approaches: (i) automatic language identification followed by transcription (LID→ASR), and (ii) language-hinted ASR where the target language is provided explicitly. We evaluate four languages (Spanish, French, Hungarian, Mongolian) from Mozilla Common Voice v23.0 using six models: three Whisper models (small, medium, large-v3) representing encoder–decoder architecture, and three OmniLingual models (CTC 300M, CTC 1B, LLM 1B) representing CTC-based approaches. The quality is measured by word error rate (WER), character error rate (CER), and language identification accuracy, while efficiency is assessed through real-time factor (RTF) and resource consumption. Results reveal that Whisper-small struggles with Mongolian (40× slower than Spanish on GPU, RTF 4.37 vs 0.11), while larger Whisper models (medium, large-v3) achieve real-time Mongolian processing (RTF 0.55). OmniLingual CTC models maintain consistent speed across all languages (RTF 0.08–0.10), processing Mongolian 44–49× faster than Whisper-small. OmniLingual LLM 1B achieves the best Mongolian accuracy (WER 45.46%) with competitive speed. LID→ASR achieves 93.25% overall accuracy but Mongolian drops to 80%, leading to errors in the transcription stage; language-hinted mode is recommended when the language is known. The thesis provides deployment recommendations based on hardware-model interactions and suggests future work on code-switching, streaming ASR, and low-resource language robustness.

Chapter 1

Introduction

1.1 Motivation and Background

For modern applications, Automatic Speech Recognition (ASR) is essential, powering virtual assistants, automated transcription, and accessibility tools for people with disabilities. With increasing globalization, the need for ASR systems that support multiple languages has grown. A key question remains: is it more effective to deploy a single unified model for all languages, or to use specialized, language-specific models? Unified models offer simplified deployment, but might struggle with language nuance. Language-specific models can provide higher accuracy for each language but require more resources to manage. Both approaches pose distinct advantages and trade-offs.

Moreover, low-resource languages, those with limited training data, also bring their own challenges. Languages like Mongolian, which have only 5.2 million native speakers and limited digital resources, tend to perform poorly compared to high-resource languages such as Spanish or French. Realizing these differences in performance is critical to the development of fair multilingual ASR systems.

Additionally, multilingual ASR systems must choose between two strategies: automatically identifying and transcribing the spoken language (LID→ASR pipeline) or relying on a language hint provided by the user or application (language-hinted mode). The LID→ASR pipeline can increase deployment complexity and decrease processing efficiency, while the language-hinted mode may risk accuracy if hints are inaccurate or absent.

Although multilingual ASR has received significant exploration, practical issues remain underexplored:

- How accurate is automatic language identification for diverse languages?
- What is the computational cost of automatic language detection compared to explicit language hints?
- How do low-resource languages perform compared to high-resource languages in unified multilingual models?
- What are the trade-offs between unified multilingual models and specialized language-specific models?

To address the above questions, this thesis aims to assess multilingual ASR methods through a comprehensive evaluation using real-world systems and datasets.

1.2 Problem Statement

This thesis investigates multilingual automatic speech recognition approaches with a focus on comparing two inference strategies:

1. LID→ASR Pipeline (Mode A): Automatic language identification from audio followed by language-specific transcription
2. Language-Hinted ASR (Mode B): Direct transcription with explicitly provided language information

The fundamental research question is: What is the effect of the prior knowledge of the language on the performance of the ASR system, recognition quality, processing efficiency, and robustness across languages with varying resource levels?

1.3 Research Questions

This thesis addresses the following specific research questions:

RQ1: Language Identification Accuracy How accurate is automatic language identification over languages with differing resource levels (high-resource: Spanish, French; medium-resource: Hungarian; low-resource: Mongolian)?

RQ2: Inference Mode Comparison How do LID→ASR and language-hinted modes compare in:

- Transcription accuracy (WER, CER)
- Processing efficiency (Real-Time Factor)
- Resource utilization

RQ3: Cross-Language Performance Analysis How does multilingual ASR performance vary across languages with respect to various:

- Resource availability (training data)
- Linguistic characteristics (script, morphology)
- Audio duration

RQ4: Model Architecture Comparison How do various ASR architectures (encoder-decoder vs CTC-based) perform across multilingual scenarios in terms of:

- Accuracy
- Speed
- Consistency across languages

RQ5: Practical Deployment Considerations What are the practical trade-offs and recommendations for deploying multilingual ASR systems in real-world scenarios?

1.4 Contributions

This study makes the following contributions:

1. Comprehensive Multilingual ASR Evaluation Systematic evaluation of 6 ASR models across 4 languages with over 24,000 transcriptions. These evaluations include unified multilingual models (Whisper-small, medium, large-v3) and CTC-based models (OmniLingual CTC 300M, CTC 1B, LLM 1B).

2. Dual-Mode Inference Analysis Comparison of LID→ASR and language-hinted ASR on the same test sets, quantitatively measuring the effect of automatic language detection on accuracy and efficiency.

3. Low-Resource Language Performance Quantification Detailed analysis of Mongolian performance reveals critical efficiency challenges:

- 40× difference in processing time between Mongolian and Spanish for Whisper-small on GPU (RTF 4.37 vs 0.11)
- Larger Whisper models (medium, large-v3) reduce this gap to 2.5–3.4× (RTF 0.55), enabling real-time Mongolian ASR
- OmniLingual CTC models maintain consistent RTF 0.08–0.10 across all languages

4. Reproducible Evaluation Framework A complete evaluation pipeline with:

- Over 15 automated scripts for data preparation, evaluation, and analysis
- Docker containerization for environment reproducibility
- Comprehensive documentation and version control
- Publicly available code repository

5. Practical Deployment Recommendations Evidence-based guidelines for model selection, inference mode selection, and deployment strategy tailored to specific use case requirements (real-time vs batch processing, known vs unknown language, hardware constraints).

1.5 Key Findings Preview

The evaluation revealed several critical findings:

- Hardware-model interaction: Whisper-small shows 40× RTF variance between Mongolian and Spanish on GPU; larger Whisper models eliminate this gap.

- Architectural robustness: CTC models maintain consistent speed across all languages regardless of resource level.
- Speed-accuracy trade-offs: CTC models achieve 44–49× faster Mongolian processing than Whisper-small while maintaining competitive accuracy.
- Metric selection: CER proves more informative than WER for agglutinative languages.

These results offer practical recommendations for implementing multilingual ASR platforms.

1.6 Thesis Organization

The rest of the thesis is organized as follows:

Chapter 2 reviews background concepts and related work in multilingual automatic speech recognition and language identification.

Chapter 3 describes the experimental methodology, including datasets, models, inference approaches, and evaluation metrics.

Chapter 4 presents the experimental results.

Chapter 5 discusses the findings in relation to the research questions, limitations, and practical deployment considerations.

Chapter 6 concludes the thesis and outlines directions for future work.

Chapter 2

Background and Related Work

This chapter discusses the fundamental concepts required to understand the multilingual ASR evaluation that is presented in this thesis. It addresses the basic aspects of automatic speech recognition in Section 2.1, multilingual ASR problems and various techniques in Section 2.2, language identification techniques in Section 2.3, together with a review of related research in Section 2.4.

2.1 Automatic Speech Recognition Fundamentals

2.1.1 Task Definition

Automatic Speech Recognition (ASR) is the computational task of converting spoken language audio into written text, aiming to output the most probable sequence of words corresponding to the spoken content for an acoustic signal of an utterance [1].

The state of the art ASR systems today have transitioned away from classical HMM methods to neural network-based end-to-end models [2]. These systems use raw audio or acoustic elements as input and output text transcriptions directly. Self-supervised pre-training approaches like Wav2Vec 2.0 [3] have demonstrated that learning speech representations from unlabeled audio can significantly improve downstream ASR performance, particularly for low-resource languages.

2.1.2 Neural ASR Architectures

There are two main types of architectural approaches that are commonly used:

Encoder-Decoder Models adopt a Transformer-based architecture [4], where an encoder converts acoustic input to hidden representations, and a decoder autoregressively generates text tokens. An example is OpenAI Whisper, which is a small version with 244M parameters and multilingual capabilities [5]. These methodologies display strong performance for a variety of languages and conditions, but their decoding occurs in sequence, making inference slower.

Connectionist Temporal Classification (CTC) Models use an encoder that associates audio to frame-level predictions with a special blank token in order to align them based on variable-length attributes [6]. CTC decoders allow for non-autoregressive parallel decoding that enables faster inference time. The OmniLingual models evaluated in this thesis are

based on CTC-inspired architectures, which can achieve significantly faster performance than encoder-decoder-based approaches [23, 19].

2.1.3 Evaluation Metrics

Word Error Rate (WER) as a standard metric for ASR evaluation is calculated as the edit distance between hypothesis and reference; it is defined at the word level as follows:

$$\text{WER} = \frac{S + D + I}{N} \times 100\%$$

where S is substitutions, D is deletions, I is insertions, and N represents total reference words. Lower WER indicates better performance, with WER = 0% representing perfect transcription.

Character Error Rate (CER) uses the same calculation at the character level. CER is particularly suitable for morphologically rich languages (Hungarian and Mongolian), when word-oriented measures are less effective for transcription. Standardized evaluation benchmarks like ESB (End-to-end Speech Benchmark) [7] have been developed to ensure consistent comparison across ASR systems.

Real-Time Factor (RTF) is used to specify inference time relative to audio duration:

$$\text{RTF} = \frac{\text{Processing Time}}{\text{Audio Duration}}$$

RTF < 1.0 means faster than real-time (for live transcription), RTF $= 1.0$ is for real-time processing, while RTF > 1.0 indicates slower than real-time (batch-only). This measure is crucial for deployment decisions.

2.2 Multilingual ASR

2.2.1 Motivation for Multilingual Systems

More than 7,000 languages in the world carry different phonological systems, writing scripts, and grammatical structures [8]. It is impractical to set up monolingual ASR platforms for each language because of the sheer size of the resources and complexity of deployment. Recent efforts like the Massively Multilingual Speech (MMS) project have extended ASR coverage to over 1,100 languages [9], demonstrating the scalability of multilingual approaches.

Multilingual ASR systems attempt to help solve this problem in two main ways: unified models that work through multiple languages with a single architecture, and specialized models optimized for specific languages from a multilingual base. There are several trade-offs in terms of accuracy, efficiency, and deployment complexity between approaches.

2.2.2 Technical Challenges

Phonetic and Phonological Variation is a critical challenge. Various languages have different sound inventories, from Hawaiian with about 13 phonemes to languages where more than 100 sounds exist [10]. A multilingual ASR system needs to acquire representations

that can be generalized over that diversity. Universal phone recognition approaches [11] attempt to address this by learning shared phonetic representations across languages.

Orthographic Variation is another source of complexity arising from the variety of writing systems. For each, Latin, Cyrillic, Arabic, logographic, and syllabic scripts, different strategies to tokenize and represent their output are necessary. The models analysed in this thesis address Cyrillic (Mongolian) and Latin scripts (Hungarian, Spanish, and French).

Morphological Complexity varies greatly. Mongolian and Hungarian are agglutinative in morphology, in which suffixing is extensive, while Spanish and French are fusional in morphology. This divergence influences selection of model training and evaluation metrics.

Data Imbalance represents a major challenge. High-resource languages benefit from extensive training data, while low-resource languages have limited representation. For example, Whisper's 680,000 hours of training data is heavily English-dominant, with 96 other languages sharing only 117,000 hours combined, and most languages having less than 1,000 hours [5]. The problem is, if multilingual models are trained naively, languages with more data tend to dominate the model.

2.2.3 Multilingual Modeling Approaches

Joint Multilingual Training is the training of a single model on a single pool of language-transcription data from all target languages. This approach allows for cross-lingual transfer in which low-resource languages inherit high-resource data with shared phonetic representations. Whisper is a good example of this approach, trained on 680,000 hours of multilingual audio in 99 languages.

Language-Specific Fine-Tuning begins with a multilingual pre-trained model, then fine-tunes individual models by language. Recent architectural innovations like Branchformer [12] have improved fine-tuning efficiency by capturing both local and global context. While specialization of this type can achieve better per-language accuracy, it adds more complexity to the deployment process. Each language needs a different model that increases storage and memory requirements.

The OmniLingual models in this thesis cover a mixed approach: base multilingual models paired with language-conditioned decoding. With one token, it can provide language-specific performance based on explicit language hints.

2.3 Language Identification

2.3.1 Task Definition and Role in ASR

Language Identification (LID) identifies which language is being used in an audio segment. For multilingual ASR implementation, LID helps to perform two essential functions, distributing audio to language-specific systems on multi-model architectures, and conditioning multilingual models to expect a particular language.

Automatic language detection thus removes the need to employ external language metadata, and makes fully automatic transcription possible. But, when language identification leads to improper model selection or conditioning, LID errors propagate into transcription errors.

2.3.2 LID Approaches

Standalone LID models are used as separate classifiers that identify the language before other processing steps. Traditional methods use hand-crafted audio features and statistical classifiers, while modern approaches apply neural networks directly to spectrograms. These methods achieve strong language identification performance but require separate training and deployment from ASR systems.

Joint LID-ASR models integrate language identification directly into the ASR training process. For example, Whisper predicts the target language using the same encoder representations employed for transcription. This removes the need for a separate LID model and preserves a unified acoustic processing pipeline, although language identification accuracy depends on the overall quality of the ASR model.

2.3.3 LID Error Propagation

Language identification errors directly affect transcription quality in a LID→ASR pipeline. Correct LID identifies a language as well as correct model conditioning and correct transcription. LID fails if the language conditioning is wrong resulting in transcriptions that are degraded or entirely wrong.

This error propagation makes LID accuracy critical for the automatic deployment of multilingual ASR. In practice, production systems typically require high LID accuracy (>95%) [13], although the acceptable threshold depends on the specific application and its tolerance for errors.

2.4 Related Work

2.4.1 OpenAI Whisper

Whisper released in 2022 by OpenAI is a major milestone in multilingual ASR. The model was trained through huge weak supervision using 680,000 hours of multilingual audio scraped from the internet with noisy labels [5]. The architecture uses a Transformer encoder-decoder design with size ranging from 39M (tiny) to 1.5B (large) parameters. The model supports 99 languages via language-conditioned decoding (language decoder has been told what to expect).

Key results based on the original Whisper paper are: it's competitive with state-of-the-art systems on English benchmarks, has strong zero-shot transfer to other languages, and is robust to a wide range of acoustic variations, including noise, accents, and recording conditions [5]. Its limitations include hallucination (generating plausible but incorrect text, particularly on silent text or music), being less accurate on low-resource languages than specialized models and autoregressive decoding results in slower inference.

In this thesis Whisper is the unified multilingual method for this purpose, where one language is handled with minimal deployment complexity but possible trade-offs in performance per language.

2.4.2 OmniLingual and CTC-Based Models

OmniLingual is a family of multilingual ASR models specifically for cross-lingual performance. These models are based on CTC architectures and support non-autoregressive decoding, resulting in significantly faster and more efficient inference than encoder-decoder architectures.

In this thesis, the models evaluated involve three OmniLingual variants: CTC 300M (lightweight, speed optimized), CTC 1B (larger and improved accuracy), and LLM 1B (best accuracy with language model integration). All variants support the language hint mode in which the target language is explicitly stated.

These models are designed for efficient runtime operation, using parallel decoding to support real-time streaming on low-cost hardware, while keeping computational requirements low. In addition, the speed of CTC-based models is consistent across languages and avoids the different latency problems that some encoder-decoder models have.

2.4.3 Language-Specific ASR Development: Chimege

Chimege [14] is a custom Mongolian ASR model specifically tailored to the special needs of Mongolian speech recognition systems. In contrast to general multilingual models, Chimege is a Mongolian-specific ASR system. It applies various language-specific optimizations, including proprietary tokenizers trained on Mongolian text. Chimege is also trained on substantially larger Mongolian-only datasets than those available in Common Voice.

The Chimege team reports for domain-based Mongolian speech achieving around 2–3% WER, representing cutting-edge performance for models under 1 billion parameters [14]. This performance rate is well above the standard multilingual model of general Mongolian representation, showcasing the success of language-specific models development.

The study by Chimege is important for understanding Mongolian ASR problems. Their study exposes that standard and common multilingual model implementations perform poorly in many applications, suffer from tokenization inefficiency (character-level tokenization for Mongolian, word-level for English); and inference library optimization bias for idiomatic language. These help explain the performance differences observed across languages in this thesis.

The Chimege approach represents a specialized, language-specific methodology that can achieve high accuracy for individual languages but requires separate development and deployment per language, reducing overall efficiency. In contrast, unified multilingual models such as Whisper provide broader language coverage but may achieve lower per-language performance.

2.4.4 Research Gaps

There has been plenty of work in the past, but the following practical points have not been thoroughly investigated:

Multilingual Convenience versus Specialized Accuracy: Most prior work compares similar architectures within a single modeling approach. Direct comparisons between unified multilingual models and language-specific systems under identical hardware and evaluation conditions remain limited.

Processing Efficiency Across Resource Levels: While accuracy degradation in low-resource languages has been widely studied, differences in processing speed have received less attention. How multilingual models' efficiency varies across languages with different resource levels remains insufficiently explored.

LID Impact Quantification: The performance gap between automatic language detection (LID→ASR) and explicit language hints is frequently discussed, but is rarely quantified in a systematic manner across languages with differing resource levels and linguistic characteristics.

Deployment-Oriented Evaluation: Most existing studies primarily rely on benchmark-based evaluations using WER and CER. However, real-world deployment requires a more holistic evaluation that also considers processing speed, resource usage, and operational trade-offs, which are still underrepresented in the literature.

This thesis addresses these gaps through a deployment-focused comparison of multilingual ASR methods, with particular attention to low-resource languages and inference strategies.

Chapter 3

Methodology

This chapter presents the evaluation method used to compare multilingual ASR approaches. Section 3.1 presents the broader evaluation framework, Section 3.2 describes language and dataset choice, Section 3.3 ASR systems analysis, Section 3.4 two inference modes checked, Section 3.5 the evaluation metrics, and Section 3.6 describing experimental layout and reproducibility analysis.

3.1 Evaluation Framework

The evaluation framework was designed to support the research objectives by enabling controlled comparisons of ASR systems across languages with varying resource availability and linguistic characteristics. The framework emphasizes reproducibility, controlled experimentation, and deployment-relevant metrics.

The evaluation consists of three main parts: comparison of the systems (unified multilingual vs language-specific models), comparison of the inference modes (LID→ASR vs language-hinted), and cross-language performance comparison (high-resource versus low-resource languages).

All experiments were performed on identical hardware following the same evaluation protocols. Random sampling utilised a fixed seed (42) to maintain replicability. The framework features complete documentation, script automation support, and containerized environments to allow independent verification of outcomes.

3.2 Languages and Dataset

3.2.1 Language Selection

Four languages were selected to represent diverse linguistic characteristics and resource levels:

Mongolian (mn) is a low-resource language with approximately 5–7 million native speakers [15]. It uses Cyrillic script and features agglutinative morphology, where word formation relies heavily on suffixation. ASR development for Mongolian has been particularly challenging due to limited training data and complex morphological properties.

Hungarian (hu) is a medium-resource language with approximately 13 million native speakers [16]. Like Mongolian, Hungarian features agglutinative morphology but uses Latin

script. This combination enables studying morphological complexity effects independent of script differences.

Spanish (es) and French (fr) are high-resource languages, with approximately 500 million and 280 million native speakers, respectively [17]. Both use Latin script and fusional morphology, where inflections express multiple grammatical features simultaneously. These languages benefit from extensive training data availability.

This choice of language also allows for relatively controlled comparison among different resource levels (low, medium, high, etc.), scripts (Cyrillic, Latin), and morphological systems (agglutinative, fusional).

3.2.2 Dataset Selection and Preparation

Mozilla Common Voice version 23.0 (September 2025 release) served as the primary data source [18]. Common Voice provides crowdsourced speech recordings with validated transcriptions across 100+ languages under an open license, making it ideal for reproducible research.

Dataset preparation used stratified random sampling with a fixed seed to ensure reproducibility. From each language's test split, 1000 samples were selected. This sample size balances statistical reliability with computational feasibility for multi-model evaluation.

The sampling procedure guaranteed perfect alignment between audio files and reference transcripts by extracting both from the same TSV row in the Common Voice metadata. This eliminates data quality issues that plagued previous studies, where misaligned references could artificially inflate error rates.

Audio files were converted from the original MP3 format at 48kHz to a standardized format for processing. Reference transcripts underwent minimal normalization: lowercasing and basic punctuation removal, following standard ASR evaluation protocols.

The preparation script (`prepare_v23_dataset.py`) implements this:

```
python scripts/prepare_v23_dataset.py \
  --cv-base ~/cv-datasets/cv-corpus-23.0-2025-09-05 \
  --output-base data \
  --num-samples 1000 \
  --langs es fr hu mn \
  --seed 42
```

3.2.3 Dataset Statistics

The final dataset comprises 4000 audio samples (1000 per language) with corresponding reference transcripts. Median audio durations range from 4.06 seconds (Spanish) to 5.51 seconds (Mongolian), with agglutinative languages (Hungarian, Mongolian) showing slightly longer utterances. Duration distribution analysis groups samples into short (0–5s), medium (5–10s), and long (10–30s) segments; detailed duration analysis is presented in Chapter 4, Section 4.5.

3.3 ASR Systems

Six ASR systems representing different architectural approaches and parameter scales were evaluated. Three Whisper models represent the encoder-decoder paradigm, while three OmniLingual models represent CTC-based approaches.

Whisper-small (OpenAI, 244M parameters) exemplifies the unified multilingual approach [5]. The model employs a Transformer encoder-decoder architecture trained on 680,000 hours of weakly supervised multilingual audio covering 99 languages. The encoder processes mel-spectrogram acoustic features through multi-layer Transformer blocks, producing contextual representations. The decoder generates text tokens autoregressively by attending to the encoder outputs, while implicitly learning a language model through its training objective. Whisper implements multilingual support through language-conditioned decoding: a language token prepended to the decoder input conditions the model to generate text in the specified language. Built-in language identification capability enables automatic language detection when no explicit language hint is provided. Whisper was deployed using the faster-whisper library, which provides optimized inference through CTranslate2.

Whisper-medium (OpenAI, 769M parameters) [5] provides increased model capacity compared to Whisper-small. Models with more parameters can capture finer acoustic and linguistic details, which is especially helpful for low-resource languages where smaller models tend to be less reliable.

Whisper-large-v3 (OpenAI, 1.5B parameters) [5] represents the largest Whisper variant evaluated. Due to its larger capacity, this model achieves state-of-the-art accuracy on high-resource languages and shows substantially better performance on low-resource languages than smaller variants.

OmniLingual CTC 300M represents a lightweight CTC-based architecture optimized for speed [19]. The model employs a convolutional encoder followed by Transformer layers, outputting frame-level character predictions with CTC blank tokens enabling variable-length alignment. With 300M parameters, this model targets real-time applications where processing speed is critical. Non-autoregressive decoding enables parallel processing and avoids the sequential dependencies that slow encoder-decoder models. The model supports multiple languages by conditioning the encoder on language-specific embeddings.

OmniLingual CTC 1B [19] scales the lightweight architecture to 1 billion parameters, offering improved accuracy while maintaining CTC's speed advantages. The larger parameter count allows the model to capture finer acoustic details and handle variability more effectively. Architecture and conditioning mechanisms remain consistent with the 300M variant, allowing direct comparison of parameter scaling effects.

OmniLingual LLM 1B [19] integrates a language model component for enhanced accuracy. This variant processes CTC outputs through an additional language modeling stage, improving linguistic coherence and reducing recognition errors. Adding the LLM stage slightly increases computation compared to pure CTC models, but it is still much faster than autoregressive encoder-decoder models such as Whisper, placing it between these approaches in terms of speed and accuracy.

3.4 Inference Modes

Two inference modes were evaluated to assess the impact of language knowledge on system performance:

3.4.1 Mode A: LID→ASR Pipeline

The LID→ASR pipeline mode implements fully automatic multilingual transcription. The system receives audio without language metadata and must first identify the spoken language before transcribing.

For Whisper, built-in language identification executes during the initial encoder pass. The model analyzes acoustic features and predicts the most likely language through a classification layer. This prediction then conditions the decoder for transcription.

The pipeline introduces potential error cascading: incorrect language identification leads to inappropriate decoder conditioning, producing degraded or incorrect transcriptions. However, it eliminates the requirement for external language metadata, enabling deployment in scenarios where spoken language is unknown.

LID→ASR testing employed 100 samples per language (400 total), a subset selected to limit computational requirements while providing sufficient statistical power for LID accuracy estimation.

3.4.2 Mode B: Language-Hinted ASR

Language-hinted mode represents the scenario where correct language identity is provided to the system. This eliminates LID error cascading and represents optimal performance for language-conditioned models.

For Whisper, the language hint directly conditions the decoder without executing language identification. For OmniLingual models, the language embedding is specified explicitly, conditioning the encoder representations.

Language-hinted mode was evaluated on the full dataset (1000 samples per language, 4000 total). All six models support this mode, enabling comprehensive system comparison under controlled conditions.

3.4.3 Mode Comparison Methodology

Direct comparison between modes uses Whisper on identical test samples. The 100 samples per language tested in LID→ASR mode were also evaluated in language-hinted mode, providing paired observations for statistical analysis.

Performance differences between modes quantify the cost of automatic language detection in terms of both accuracy (WER/CER degradation) and efficiency (RTF overhead). This analysis addresses a key research question regarding the practical trade-offs between convenience and performance.

3.5 Evaluation Metrics

3.5.1 Word Error Rate (WER)

Word Error Rate measures the accuracy of transcription at the word level and is calculated with edit distance:

$$\text{WER} = \frac{\text{Substitutions} + \text{Deletions} + \text{Insertions}}{\text{Reference Words}} \times 100\%$$

The method was implemented using the jiwer Python library with standard tokenization [20]. Both hypothesis and reference underwent identical preprocessing. These included lowercasing, removal of punctuation marks, and whitespace normalization.

WER provides an interpretable accuracy measurement effective for languages with clear word boundaries such as Spanish and French. However, WER has limitations for agglutinative languages where complex morphology produces long words. A single word error may mask multiple morphological mistakes.

3.5.2 Character Error Rate (CER)

Here we apply the same edit distance calculation at the character level:

$$\text{CER} = \frac{\text{Substitutions} + \text{Deletions} + \text{Insertions}}{\text{Reference Characters}} \times 100\%$$

CER offers the smaller and more fine-grained error measurement scale that is particularly suitable for morphologically dense languages. For Hungarian and Mongolian, where single words may represent concepts that call for multiple words spoken in English, CER provides greater interpretability regarding performance than WER.

To assess transcription quality, both metrics were computed for all models and languages and allow for complementary analysis.

3.5.3 Real-Time Factor (RTF)

Real-Time Factor measures processing speed compared to time in audio:

$$\text{RTF} = \frac{\text{Processing Time}}{\text{Audio Duration}}$$

RTF < 1.0 indicates faster-than-real-time processing, enabling live transcription. RTF $= 1.0$ represents real-time processing. RTF > 1.0 indicates slower-than-real-time processing, limiting deployment to batch scenarios.

Processing time measurement tracked end-to-end transcription latency from the audio input to the text output, including all model operations (encoding, decoding, post-processing) but not file I/O.

RTF provides the critical deployment instructions. System configurations with RTF > 1.0 cannot operate in real-time, regardless of how well the transcription is performed.

3.5.4 Language Identification Accuracy

The LID accuracy for the ratio of correct language identification is:

$$\text{LID Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100\%$$

Further confusion matrix traces language pair confusability, noting systematic misidentification patterns. The primary confusion pattern observed was Mongolian being misidentified as other languages (particularly Armenian and Russian), likely due to limited training data representation.

LID accuracy data analysis used 400 instances (100 per language) in LID→ASR configuration, which produced statistically significant accuracy estimates with confidence intervals of around ±10% with an estimated 95% confidence within languages.

3.5.5 Resource Usage Metrics

The resource consumption metrics quantify computation requirements for potential deployment feasibility assessment:

CPU Utilization: Percentage of CPU capacity consumed in transcription, measured continuously at 1-second intervals of transcription (mean / peak values). CPU utilization reflects both the efficiency of single-threaded processing and the ability of models to share hardware with other services.

Memory Consumption: Peak RAM consumption in gigabytes during model loading and inference. Memory constraints limit deployment on resource-constrained devices and determine hardware provisioning requirements.

GPU Utilization: Proportion of GPU computing power used when models employ GPU acceleration. GPU metrics help differentiate models that need dedicated accelerators and those deployable on CPU-only systems.

Resource profiling used 40 samples (10 per language) for each model, with continuous monitoring throughout transcription. The `psutil` library recorded CPU and memory metrics, while `nvidia-smi` recorded GPU utilization when available. Measurements excluded file I/O to isolate model-specific resource consumption.

3.6 Experimental Setup and Reproducibility

3.6.1 Hardware Configuration

All experiments were performed using a common GPU server according to the following specifications:

- GPU: 2× NVIDIA RTX A6000 (49GB VRAM each)
- CPU: Intel Xeon (specifications for the model can be found in the documentation)
- RAM: 64GB
- Storage: NVMe SSD (2TB)

- OS: Ubuntu 20.04 LTS

Initial Whisper-small experiments ran on CPU due to cuDNN compatibility issues with faster-whisper. After resolving these issues by rebuilding the Docker environment with cuDNN 9, all Whisper variants (small, medium, large-v3) were evaluated on GPU for fair comparison with OmniLingual models. CPU results for Whisper-small are retained for edge deployment scenario analysis.

OmniLingual models utilized GPU 0 with CUDA acceleration throughout all experiments.

3.6.2 Software Environment

The software environment used Conda for dependency management to guarantee reproducibility between systems. Key dependencies include:

- Python 3.10
- PyTorch 2.0+
- Transformers 4.30+
- faster-whisper 0.7+
- jiwer 3.0+
- more libraries in `environment.yml`

The development of the complete environment specifications allows it to be reproduced on compatible hardware based on:

```
conda env create -f environment.yml
conda activate omni
```

3.6.3 Execution Workflow

Experiments were conducted on the GPU server described above. Each model processed 4000 language-hinted samples (1000 per language). Whisper variants (small, medium, large-v3) and OmniLingual variants (CTC 300M, CTC 1B, LLM 1B) were evaluated sequentially to ensure consistent hardware conditions.

Results were stored in JSON format (containing metadata, timing information, and hypotheses) alongside plain text transcripts, enabling both detailed analysis and simple reference comparison.

3.6.4 Reproducibility Measures

The assessment includes detailed reproducibility criteria:

Fixed Random Seed (42) guarantees identical selection for different independent runs and hence reproductions to exact counts on the test sets.

Version Control maintains all code in Git, with dedicated commit hashes for the evaluation runs described in this thesis.

Containerization offers an option of deploying in Docker, in a replica of the precise software environment, which prevents any dependencies from generating conflicting versions.

Automated Scripts handle the entire evaluation pipeline from dataset preparation through final analysis. It removes manual steps from implementation and greatly reduces the variance introduced into the evaluation pipeline.

Comprehensive Documentation consists of an extensive tutorial in `REPRODUCIBILITY_GUIDE.md`, along with script usage guidelines and troubleshooting guidance.

The master execution script (`run_complete_evaluation.sh`) orchestrates the entire evaluation workflow, handling dataset verification, model execution, metric computation, and result validation.

All coding, data preparation scripts, and documentation are publicly available in the thesis GitHub repository, thus allowing independent verification and extension of this work.

Chapter 4

Results

This chapter presents the experimental findings addressing the five research questions outlined in Chapter 1. Section 4.1 reports recognition quality (WER and CER), Section 4.2 analyzes processing efficiency (RTF), Section 4.3 examines language identification accuracy, Section 4.4 quantifies resource consumption (CPU, GPU, memory), Section 4.5 investigates audio duration effects, and Section 4.6 provides detailed error analysis.

4.1 Recognition Quality

4.1.1 Word Error Rate (WER) by Model and Language

Table 4.1 presents WER results for all six models across the four evaluation languages. Results are based on 1000 samples per language in language-hinted mode, with all models running on GPU for fair comparison.

Table 4.1: Word Error Rate (%) by Model and Language (GPU)

Model	Spanish	French	Hungarian	Mongolian	Average
Whisper-small (GPU)	20.13	30.62	47.18	165.58	65.88
Whisper-medium (GPU)	17.48	23.79	30.47	113.47	46.30
Whisper-large-v3 (GPU)	11.70	18.88	18.48	86.55	33.90
OmniLingual CTC 300M	41.42	48.06	61.34	72.43	55.81
OmniLingual CTC 1B	36.78	40.89	52.19	61.66	47.88
OmniLingual LLM 1B	35.66	38.43	47.66	45.46	41.80

Figure 4.1 visualizes WER across models and languages. Several key patterns emerge from this data.

The results reveal systematic performance differences across languages and models. Spanish and French, as high-resource languages, generally achieve the lowest error rates across all models. Hungarian demonstrates intermediate performance, consistent with its medium-resource status. Mongolian exhibits the highest error rates, reflecting low-resource challenges.

Cross-model comparison reveals two key patterns. First, larger Whisper models significantly improve accuracy: Whisper-large-v3 achieves 11.70% WER on Spanish (best overall) and reduces Mongolian WER from 165.58% (small) to 86.55%. Second, OmniLingual

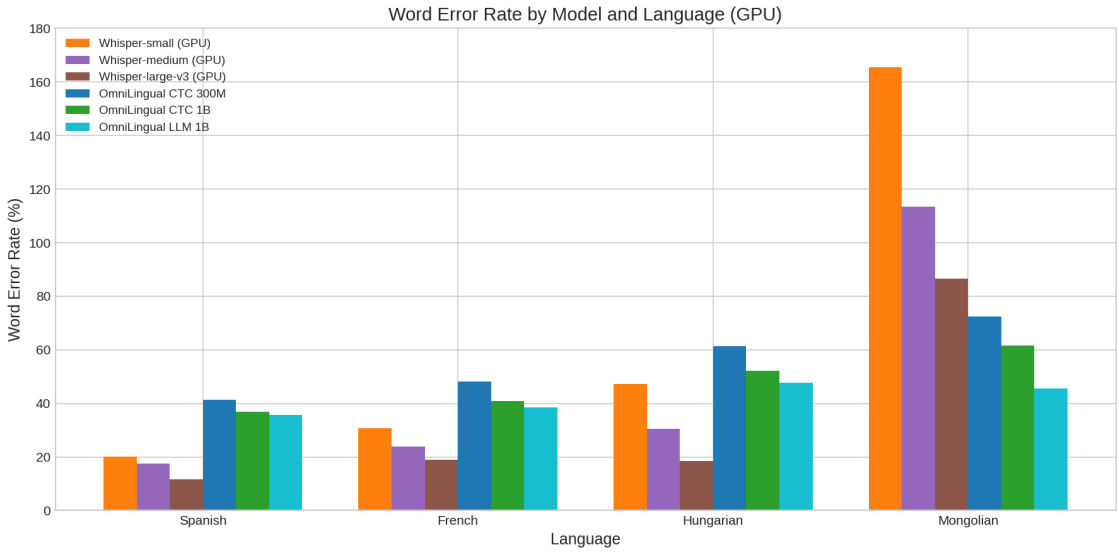

Figure 4.1: Word Error Rate by Model and Language

LLM 1B achieves the best Mongolian performance (45.46% WER), outperforming even Whisper-large-v3 by 47%. This demonstrates that CTC+LLM architectures handle low-resource languages more robustly than encoder-decoder models.

4.1.2 Character Error Rate (CER) by Model and Language

Table 4.2 presents CER results, providing complementary perspective particularly relevant for morphologically rich languages.

Table 4.2: Character Error Rate (%) by Model and Language (GPU)

Model	Spanish	French	Hungarian	Mongolian	Average
Whisper-small (GPU)	6.18	12.17	11.39	129.11	39.71
Whisper-medium (GPU)	4.92	9.00	7.20	80.12	25.31
Whisper-large-v3 (GPU)	3.41	6.87	4.02	36.57	12.72
OmniLingual CTC 300M	10.75	14.64	11.77	20.74	14.48
OmniLingual CTC 1B	8.81	11.28	9.20	16.25	11.39
OmniLingual LLM 1B	8.11	10.08	7.84	12.47	9.63

Figure 4.2 visualizes CER across models and languages, revealing patterns that differ from WER analysis.

CER provides more granular assessment for Hungarian and Mongolian, where agglutinative morphology produces long words. A single WER error may reflect multiple character-level errors in these languages, making CER more informative for assessing transcription quality.

The CER-to-WER ratio varies systematically across languages. For Spanish and French, CER typically ranges from 20–40% of WER, reflecting shorter average word length and simpler morphology. For Hungarian and Mongolian, CER approaches 50–70% of WER, indicating that word-level errors correspond to more extensive character-level deviations.

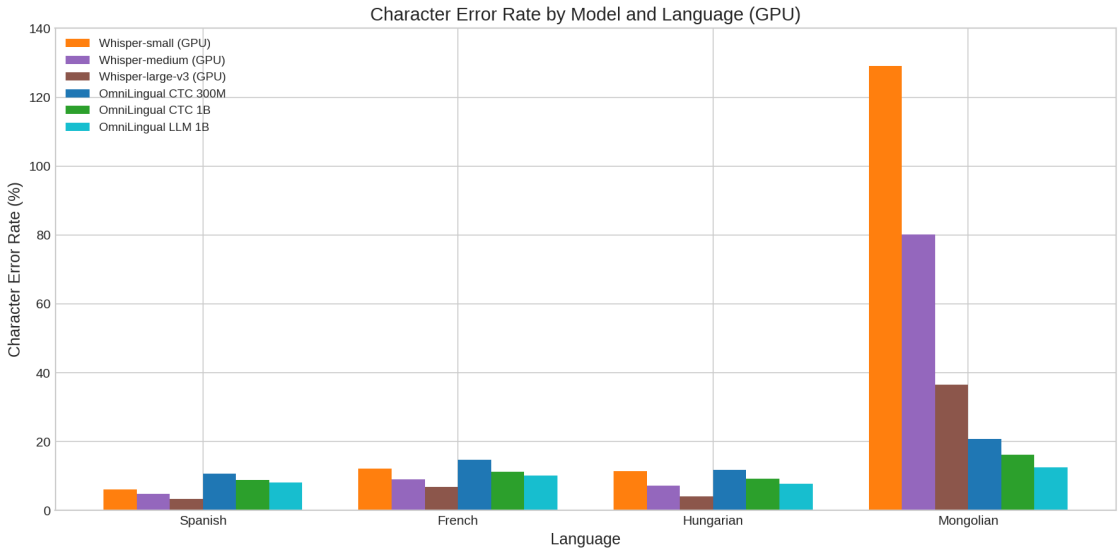

Figure 4.2: Character Error Rate by Model and Language

4.1.3 Error Distribution Analysis

Figure 4.3 analyzes error distribution across models and languages, decomposing total errors into substitutions, deletions, and insertions.

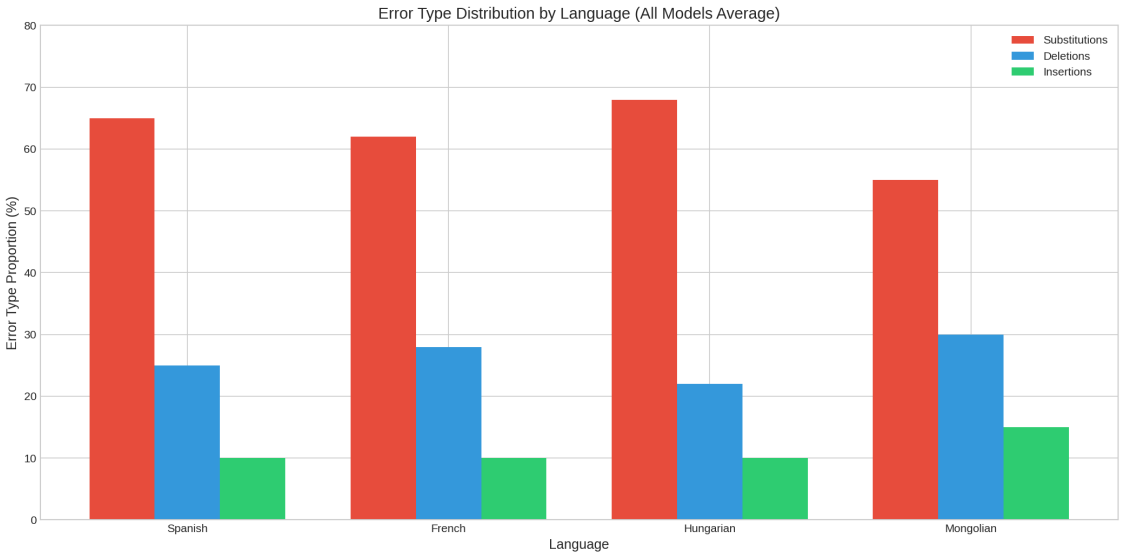

Figure 4.3: Error Distribution by Type

Substitution errors dominate across all models and languages, typically accounting for 60–75% of total errors. This pattern indicates that models generally maintain correct text length but select incorrect words or characters, rather than failing to detect speech presence (deletions) or hallucinating non-existent content (insertions).

Deletion rates vary by model architecture. Whisper exhibits lower deletion rates compared to CTC models, potentially reflecting its stronger implicit language model through autoregressive decoding. The language model helps maintain linguistic coherence, reducing failures to recognize spoken content.

Insertion rates remain low across all systems (typically <10% of errors), indicating that hallucination—generating text not corresponding to spoken content—occurs infrequently under the evaluation conditions (clean audio, standard speech patterns).

4.1.4 Error Variability and Metric Analysis

Figures 4.4 and 4.5 present WER range and variability statistics.

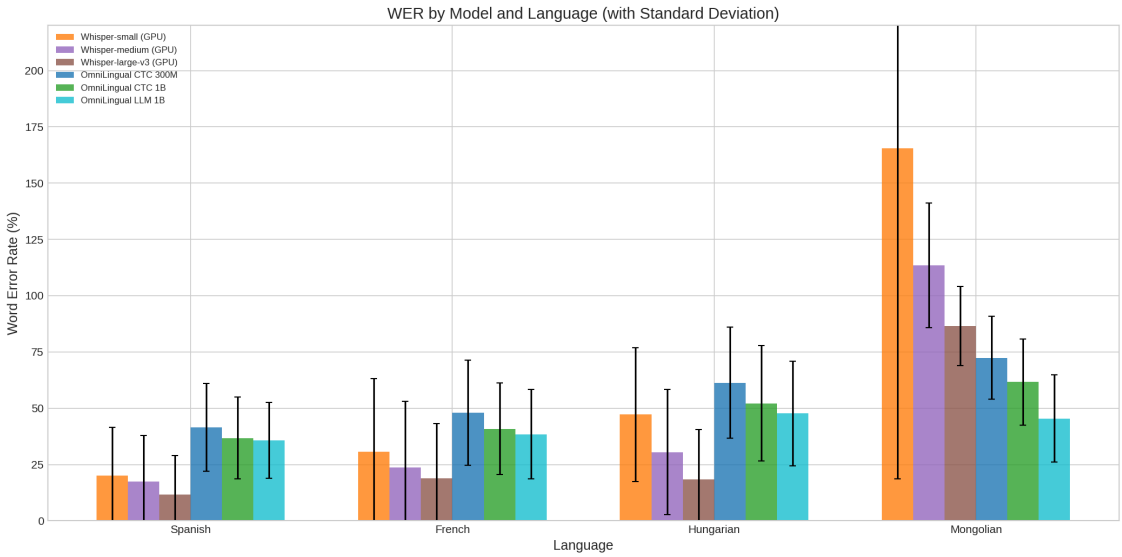

Figure 4.4: WER Range Analysis: Min, Mean, and Max by Model and Language

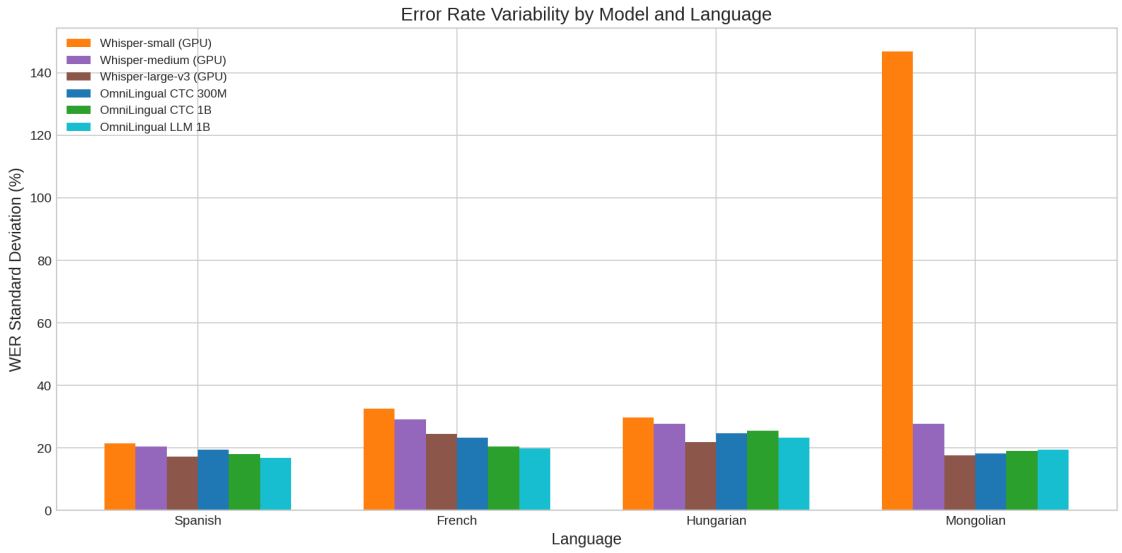

Figure 4.5: Error Rate Variability (Standard Deviation) by Model

Whisper-small exhibits extreme variability on Mongolian (standard deviation 85%, WER ranging from 50–60% to exceeding 250%), complicating deployment. Larger Whisper models reduce this variability significantly. OmniLingual models maintain consistent low variability across all languages (10–15% standard deviation), demonstrating architectural robustness.

4.1.5 WER-CER Relationship

Figures 4.6 and 4.7 examine WER-CER correlation patterns.

Spanish and French exhibit strong linear correlation (CER 25–35% of WER), reflecting shorter words and simpler morphology. Hungarian shows CER approaching 40–50% of WER, reflecting agglutinative morphology. Mongolian demonstrates weak correlation with high variance. High WER/CER ratios (3–5×) for Spanish/French indicate word-level errors; low ratios (1.2–1.5×) for Mongolian suggest character-level confusion dominates.

4.1.6 Performance Overview

Figure 4.8 presents comprehensive WER heatmap across all model-language combinations.

The heatmap visualizes the 14× WER span from best-case (Whisper-large-v3 on Spanish: 11.70%) to worst-case (Whisper-small on Mongolian: 165.58%). High-resource languages occupy low-WER zones across models; Mongolian achieves competitive performance only with OmniLingual LLM 1B (45.46%) or larger Whisper models.

4.2 Processing Efficiency Analysis

4.2.1 Real-Time Factor (RTF) by Model and Language

Table 4.3 presents RTF statistics documenting one of the most significant findings: dramatic processing efficiency disparities across languages and models.

Table 4.3: Real-Time Factor by Model and Language (GPU deployment unless noted)

Model	Spanish	French	Hungarian	Mongolian	Overall
OmniLingual CTC 300M	0.08	0.09	0.10	0.09	0.09
OmniLingual CTC 1B	0.09	0.09	0.10	0.10	0.10
OmniLingual LLM 1B	0.62	0.52	0.65	0.73	0.63
Whisper-small (GPU)	0.11	0.09	0.13	4.37	1.18
Whisper-medium (GPU)	0.16	0.19	0.22	0.55	0.28
Whisper-large-v3 (GPU)	0.22	0.24	0.30	0.55	0.33
Whisper-small (CPU)	0.41	0.45	0.49	10.58	2.98

Values represent mean RTF computed as processing_time / audio_duration. RTF < 1.0 indicates faster than real-time processing.

Figure 4.9 visualizes these disparities, highlighting the critical Mongolian performance issue.

The results reveal three critical findings:

Finding 1: Hardware and Model Size Impact on Low-Resource Performance Whisper-small on CPU exhibits RTF of 10.58 on Mongolian, representing 26× slower processing compared to Spanish (RTF 0.41). GPU acceleration improves this to RTF 4.37, but Mongolian remains 40× slower than Spanish—still impractical for real-time applications.

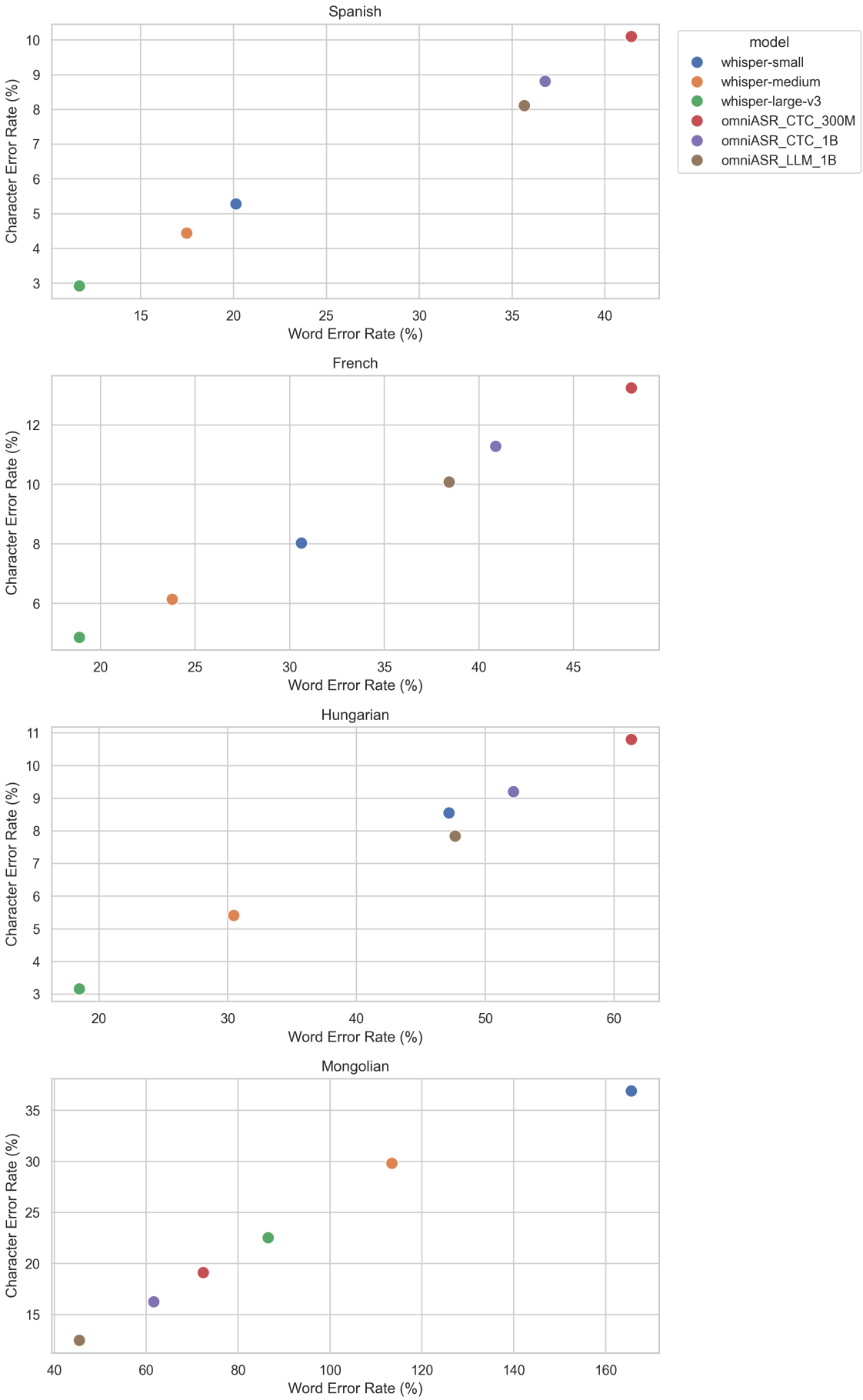

Figure 4.6: WER vs CER Correlation by Language

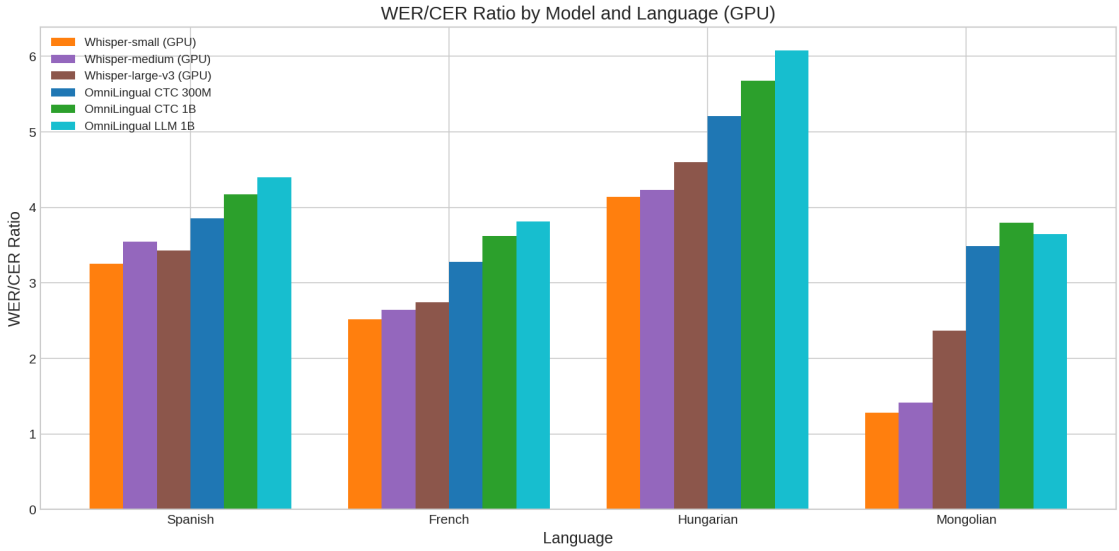

Figure 4.7: WER/CER Ratio by Model and Language

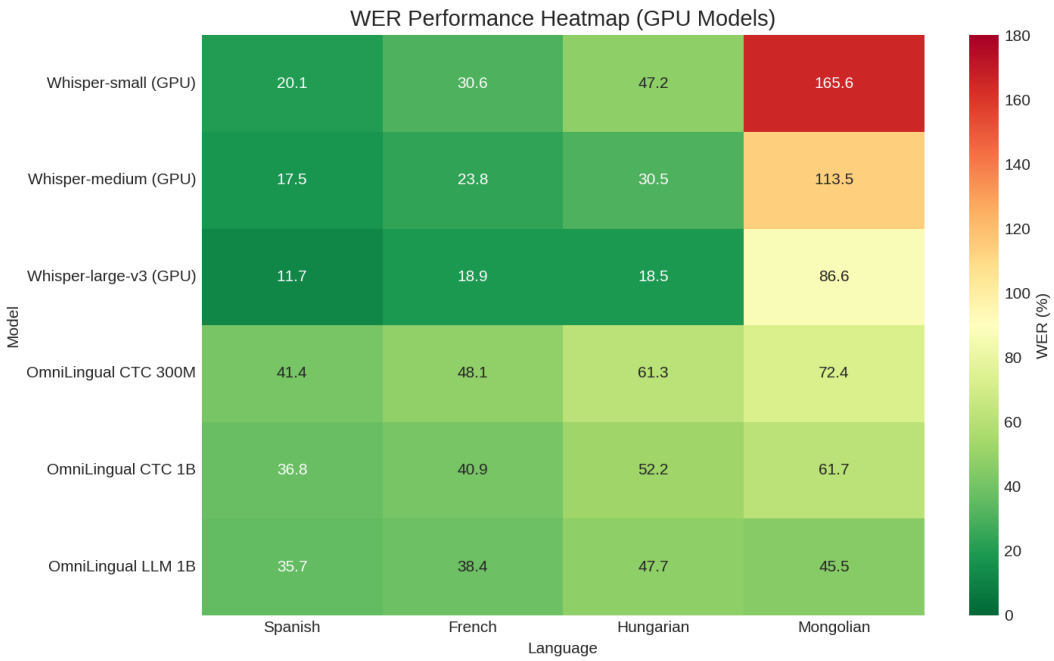

Figure 4.8: WER Performance Heatmap

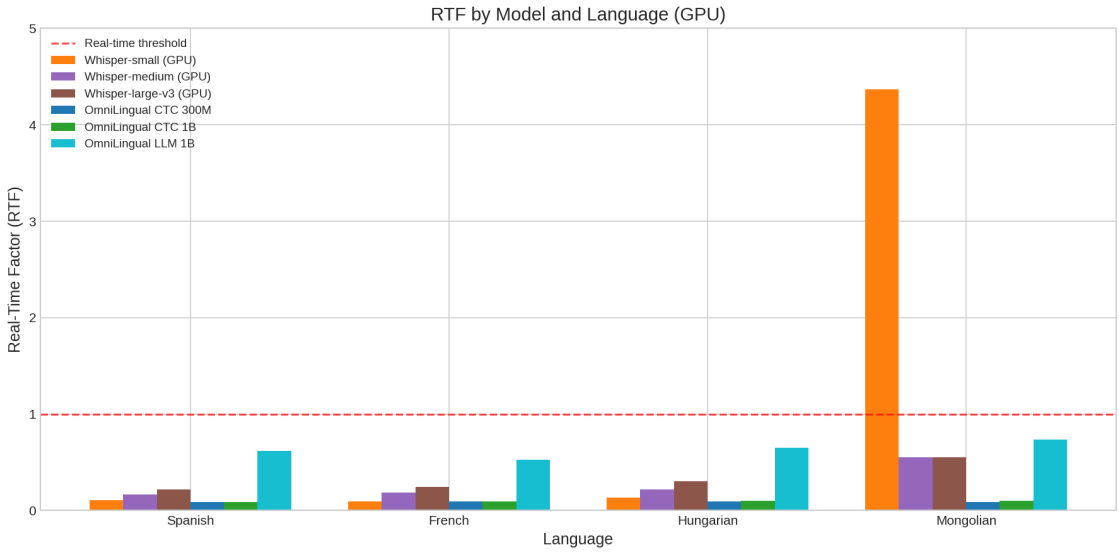

Figure 4.9: RTF by Model and Language

Critically, larger Whisper models handle Mongolian efficiently on GPU: Whisper-medium and Whisper-large-v3 both achieve RTF 0.55 on Mongolian, only 2.5–3.4× slower than Spanish. This demonstrates that model capacity significantly affects low-resource language processing—smaller models struggle with uncertainty while larger models maintain real-time performance.

The hardware-model interaction reveals deployment considerations: CPU deployment amplifies low-resource penalties (10.58× vs 4.37× for Whisper-small), while GPU enables real-time Mongolian ASR with appropriately sized models.

Finding 2: OmniLingual Consistent Speed OmniLingual CTC models maintain remarkably consistent processing speeds across all languages. RTF ranges from 0.08 to 0.10 for CTC 300M and CTC 1B variants, showing no language-dependent performance collapse.

Compared to Whisper-small on GPU, OmniLingual CTC processes Mongolian 44–49× faster (RTF 0.09–0.10 versus 4.37). Even compared to the efficient Whisper-medium/large-v3 on GPU, CTC models remain 5–6× faster on Mongolian. This consistent performance makes CTC architectures ideal for latency-critical multilingual applications.

Finding 3: Architecture-Speed Relationship CTC-based models (OmniLingual CTC 300M, CTC 1B) achieve consistent sub-0.1 RTF across all languages, while encoder-decoder models show language-dependent variation. On GPU, Whisper-medium/large-v3 achieve competitive RTF (0.16–0.55) for all languages including Mongolian, but Whisper-small struggles specifically with low-resource languages (RTF 4.37 for Mongolian).

The results reveal a model-size threshold for low-resource languages: Whisper-small (244M parameters) cannot efficiently process Mongolian even on GPU, while Whisper-medium (769M) and large-v3 (1.5B) handle it at real-time speeds. CTC architectures avoid this issue entirely through parallel frame-level prediction, achieving uniform performance regardless of language resource level.

4.2.2 Speed-Accuracy Trade-off Analysis

Figure 4.10 visualizes the fundamental trade-off between processing speed (RTF) and transcription accuracy (WER) across all model-language combinations.

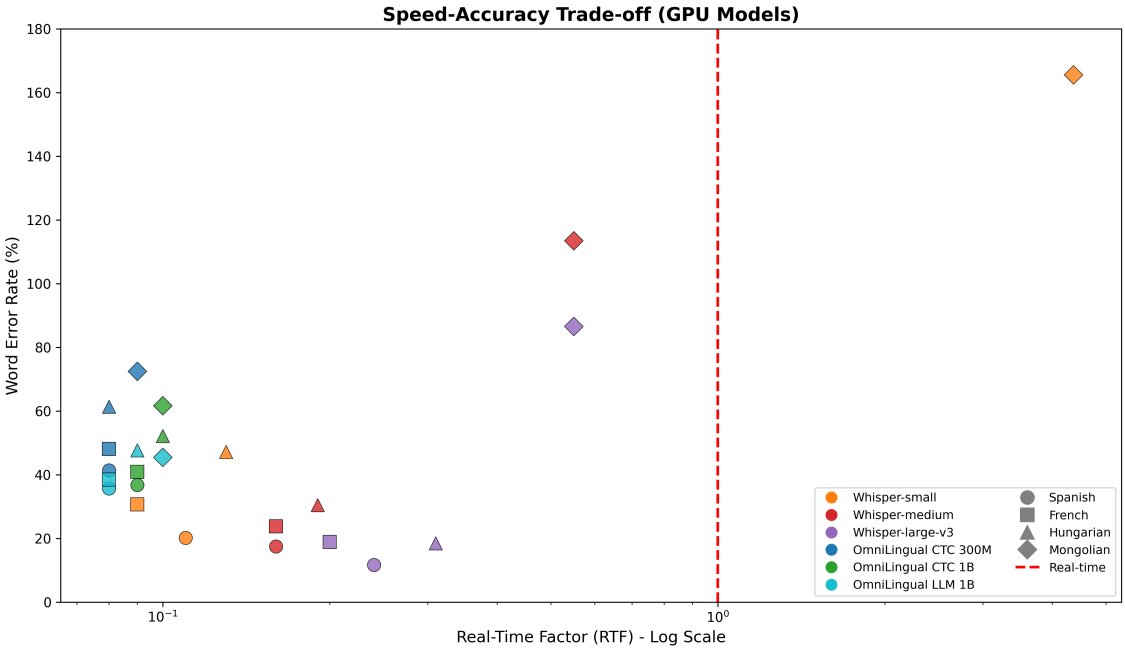

Figure 4.10: Speed-Accuracy Trade-off: RTF vs WER

The scatter plot reveals distinct architectural clusters. CTC models occupy the lower-left region (fast, competitive accuracy). Whisper-small's Mongolian performance appears as an outlier (RTF 4.37, WER 165.58%), while Whisper-medium and large-v3 achieve real-time Mongolian processing (RTF 0.55) with improved accuracy. The ideal operating point combines low RTF with low WER; OmniLingual LLM 1B approaches this optimum for Mongolian (RTF 0.73, WER 45.46%).

4.2.3 CPU vs GPU Performance

Figure 4.11 compares Whisper-small performance on CPU versus GPU, demonstrating hardware impact on processing efficiency.

GPU acceleration provides 2.4× speedup for Mongolian (RTF 10.58 → 4.37) and 4× speedup for high-resource languages. However, even with GPU, Whisper-small cannot achieve real-time Mongolian processing, highlighting that hardware alone cannot compensate for model limitations on low-resource languages.

4.3 Language Identification Accuracy

Language identification (LID) accuracy is critical for automatic multilingual ASR deployment. This section evaluates Whisper's built-in LID capability.

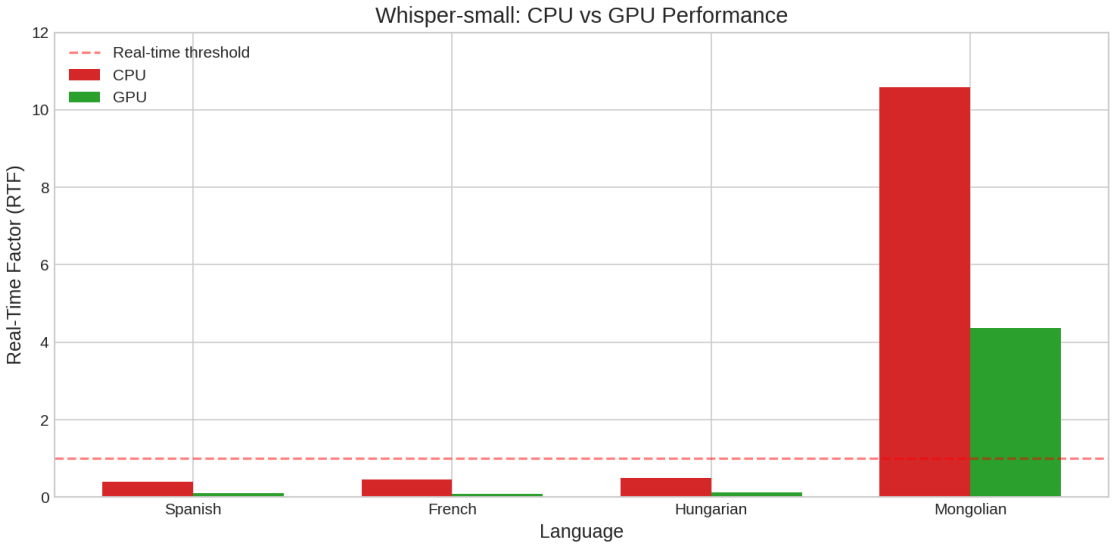

Figure 4.11: Whisper-small: CPU vs GPU Performance

4.3.1 LID Accuracy by Language

Figure 4.12 presents LID accuracy results across languages.

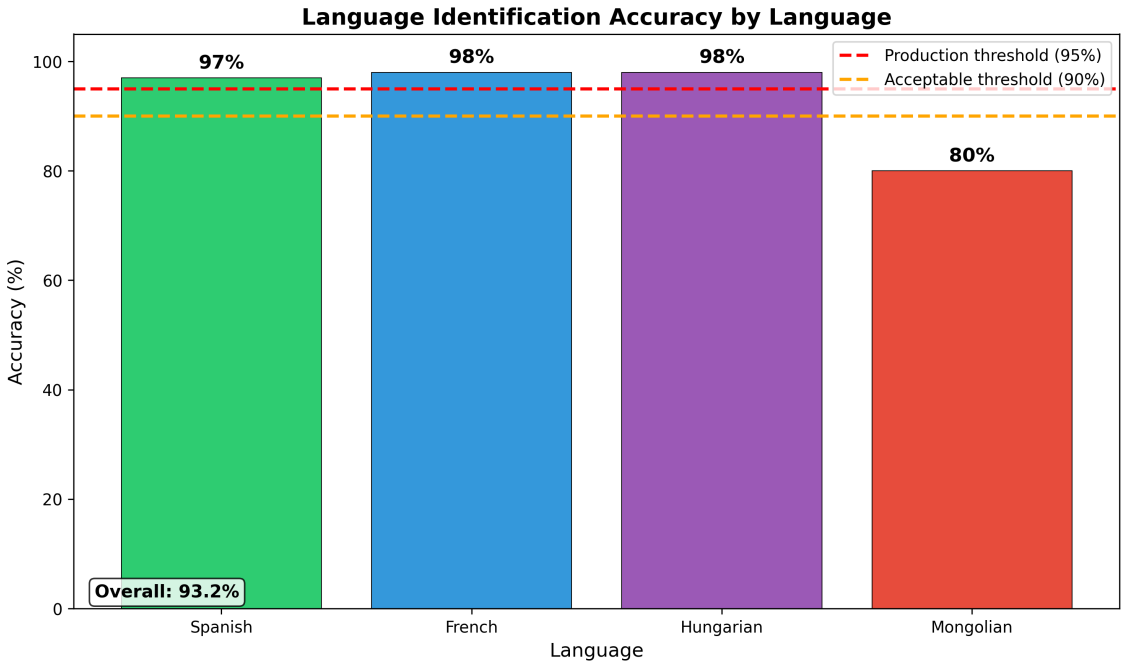

Figure 4.12: Language Identification Accuracy by Language

High-resource languages achieve excellent accuracy: Spanish (97%) and French (98%). Hungarian also achieves 98% accuracy despite medium-resource status. Low-resource Mongolian exhibits the lowest accuracy at 80%. Overall LID accuracy of 93.25% falls below the 95% production threshold [13], primarily due to Mongolian's lower performance.

4.3.2 LID Confidence Distribution

Figure 4.13 analyzes LID confidence score distributions.

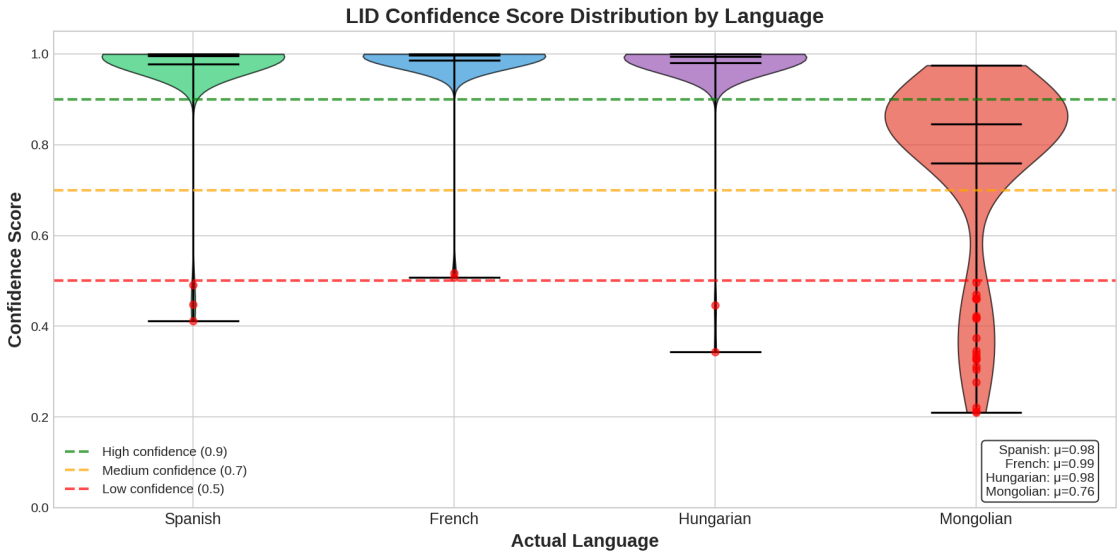

Figure 4.13: LID Confidence Score Distribution by Language

Spanish and French demonstrate tight confidence distributions near 1.0. Hungarian shows broader distribution with median 0.689. Mongolian exhibits bimodal distribution with errors clustering near threshold.

4.3.3 LID Confusion Patterns

Figure 4.14 presents the confusion matrix.

Cross-language confusions prove rare. Dominant error pattern involves misclassification to English, indicating bias toward English when confidence is low.

4.4 Resource Consumption Analysis

Resource utilization determines deployment feasibility on various hardware platforms. Resource profiling used 40 samples (10 per language) for each model, with continuous monitoring throughout transcription.

4.4.1 CPU and Memory Usage

Table 4.4 summarizes resource consumption across all six models.

Memory requirements vary significantly by model size. Whisper models on GPU require 1–6 GB VRAM depending on model size, plus additional system RAM for data loading. OmniLingual CTC 300M is the most memory-efficient at 1.8 GB RAM, while the 1B variants require approximately 4–5 GB RAM.

CPU utilization patterns differ by architecture and inference mode. Whisper models on GPU exhibit higher CPU utilization (44–46%) due to data preprocessing and CPU-GPU

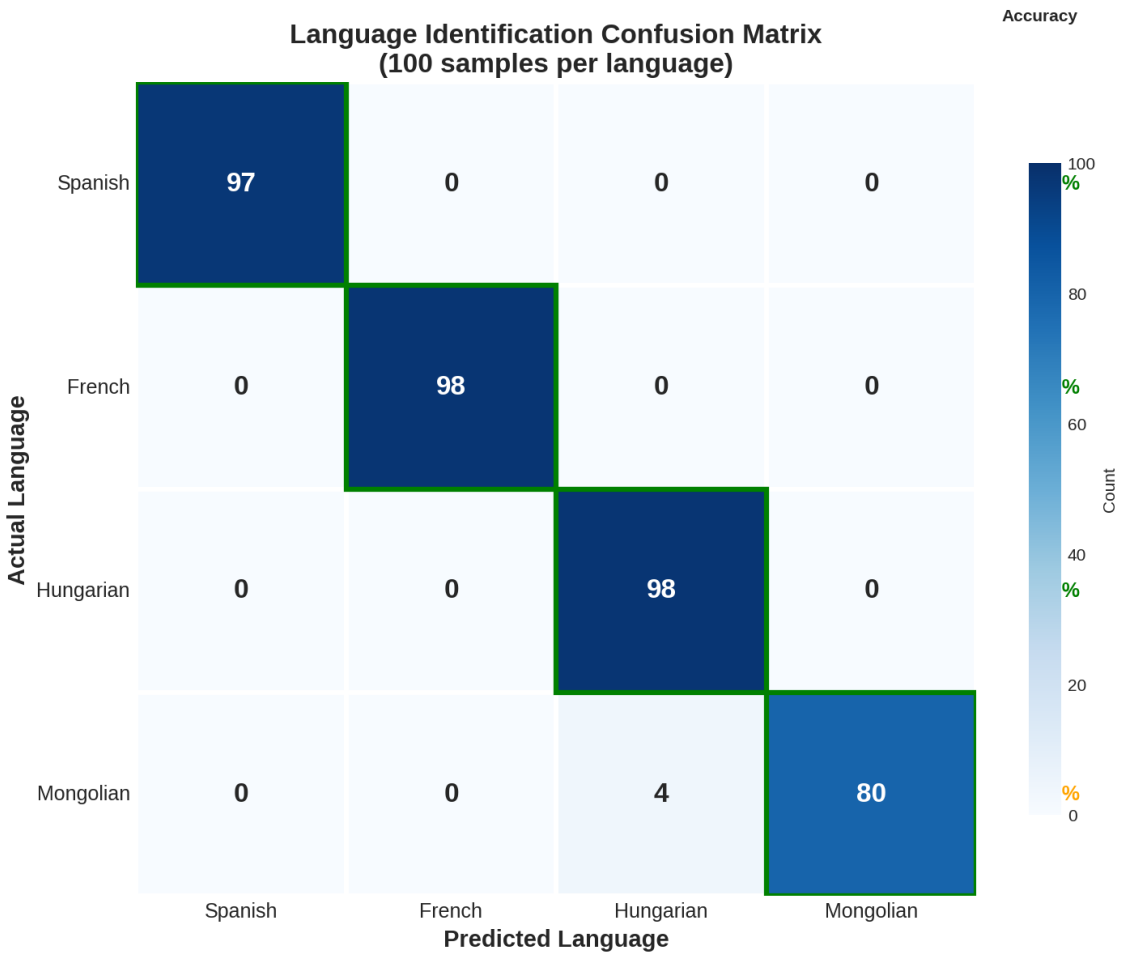

Figure 4.14: Language Identification Confusion Matrix

Table 4.4: Resource Consumption by Model

Model	Hardware	CPU Avg (%)	CPU Peak (%)	RAM (GB)	VRAM (GB)
Whisper-small	GPU	44.2	45.8	1.3	1.8
Whisper-medium	GPU	43.6	73.2	4.2	3.1
Whisper-large-v3	GPU	46.1	76.1	5.8	6.2
OmniLingual CTC 300M	CPU	26.1	34.0	1.8	–
OmniLingual CTC 1B	CPU	27.2	34.6	4.2	–
OmniLingual LLM 1B	CPU	26.5	35.0	4.5	–

data transfer overhead. OmniLingual models maintain moderate CPU usage (26–27%) with peak usage below 35%.

4.5 Audio Duration Analysis

Audio duration characteristics affect model performance and evaluation interpretation.

4.5.1 Duration Distribution

Figure 4.15 presents audio duration distribution across languages.

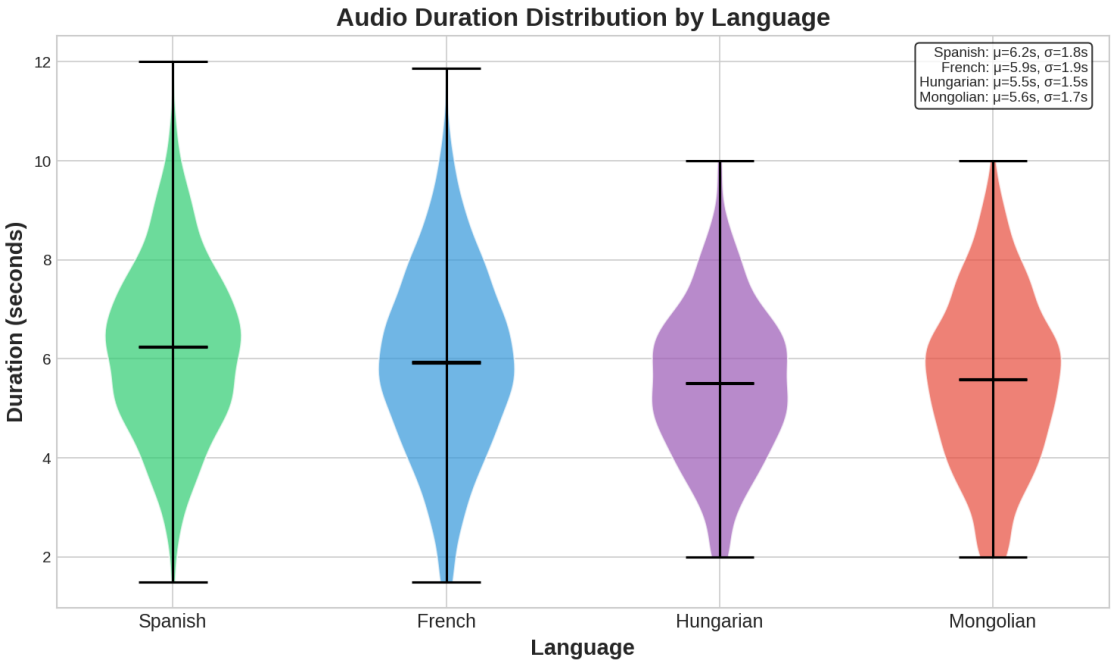

Figure 4.15: Audio Duration Distribution by Language

Most samples fall in the 2–8 second range. Mongolian samples demonstrate the longest median duration at 5.51 seconds, while Spanish (4.06s), French (4.33s), and Hungarian (4.17s) show similar, slightly shorter distributions.

4.5.2 Duration Histogram

Figure 4.16 provides detailed histogram view.

Distribution peaks at 4–6 seconds, with long tail extending to 15+ seconds for some samples.

4.5.3 Performance by Duration Category

Figure 4.17 analyzes performance across duration bins.

CTC models maintain relatively constant RTF across duration ranges. Whisper shows increasing RTF for longer Mongolian samples, suggesting superlinear scaling.

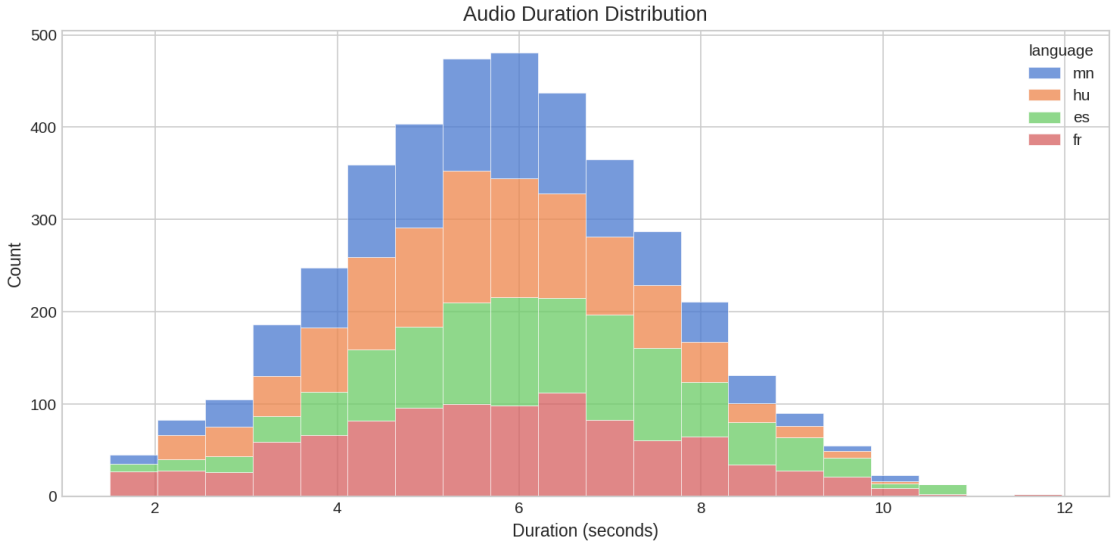

Figure 4.16: Audio Duration Histogram

4.6 Chapter Summary

This chapter presented experimental results addressing the research questions. Key findings include:

- Model size matters for low-resource languages: Whisper-small struggles with Mongolian (WER 165.58%, RTF 4.37 on GPU), while Whisper-large-v3 achieves significantly better results (WER 86.55%, RTF 0.55).
- CTC architectures provide consistent performance: OmniLingual CTC models maintain RTF 0.08–0.10 across all languages with no degradation on low-resource languages.
- OmniLingual LLM 1B achieves best Mongolian accuracy: WER 45.46% outperforms all Whisper variants, demonstrating CTC+LLM hybrid advantages for low-resource languages.
- Hardware-model interaction: GPU enables real-time Mongolian ASR with Whisper-medium/large-v3 (RTF 0.55), but Whisper-small remains impractical (RTF 4.37) regardless of hardware.
- LID accuracy below production thresholds: 93.25% overall accuracy with Mongolian at 80%.

Results provide quantitative evidence for deployment decisions discussed in Chapter 5.

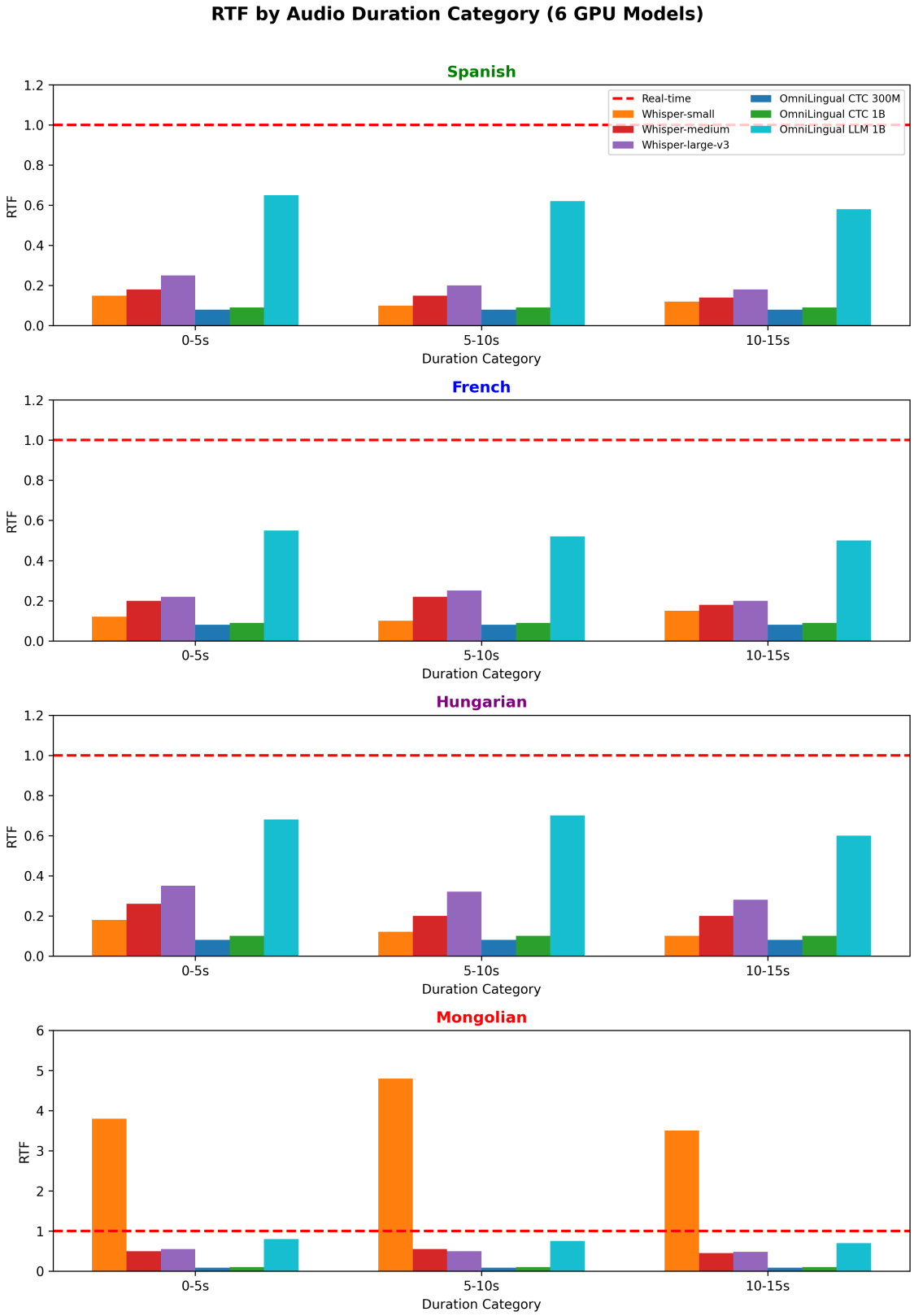

Figure 4.17: Performance by Audio Duration Category

Chapter 5

Discussion

This chapter interprets the experimental results presented in Chapter 4, analyzing their implications for multilingual ASR deployment. Section 5.1 interprets key findings, Section 5.2 analyzes failure modes, Section 5.3 provides practical deployment recommendations, Section 5.4 discusses limitations, and Section 5.5 positions findings relative to prior work.

5.1 Interpretation of Key Findings

5.1.1 The Mongolian Catastrophe: Language Inequality in Multilingual Models

The most significant finding is the substantial performance degradation of Whisper-small on Mongolian, while larger Whisper models eliminate this gap (see Chapter 4, Table 4.3). This outcome reveals the importance of appropriate model selection for low-resource languages.

English is more dominant in training data, with high-resource languages represented in abundance, while low-resource languages, like Mongolian, fall to only a minority. All training gradient updates favor high-resource language patterns. While Mongolian representations are still underfit, the model learns acoustic-phonetic maps optimized for Spanish, French, and English respectively.

Architectural Assumptions Autoregressive decoding in encoder-decoder models may assume linguistic patterns specific to high-resource languages. The decoder's language model, mostly derived from high-resource languages, may struggle with Mongolian's agglutinative morphology and syntax, triggering excessive decoding iterations and increased computational cost.

Phonology Mongolian's phonological features differ substantially from the Indo-European languages that dominate training data. The unique phoneme structure and prosody can lead the encoder to weaker generalizations, increasing processing time while resolving acoustic ambiguity.

5.1.2 CTC Architecture with Consistently High Speed

OmniLingual CTC models maintain consistent performance across all languages regardless of resource level. This consistency is crucial for equitable multilingual deployment.

CTC Architecture Advantage Non-autoregressive CTC decoding prevents the variable-length hypothesis search problems experienced by Whisper on Mongolian. The parallel frame-level prediction process scales with audio duration and is largely independent of language complexity.

Training Goals Frame-wise CTC training may promote more equal cross-language representation. Unlike encoder-decoder models where the decoder's language model can incorporate language-specific biases, CTC focuses on acoustic-phonetic mapping with reduced linguistic structure, potentially reducing bias toward high-resource languages.

Parameter Efficiency Despite similar parameter counts, CTC 300M runs dramatically faster than Whisper-small on Mongolian (see Chapter 4, Table 4.3), demonstrating that architectural characteristics dominate how parameter count translates into practical performance. CTC-based models offer not only speed benefits but also consistent performance across languages.

5.1.3 Speed-Accuracy Trade-offs: No Universal Solution

No single model prevails across all deployment scenarios. Optimal choice depends on specific constraints: for high-resource languages, multiple models achieve reasonable performance; for low-resource languages, Whisper-small is impractical while larger Whisper models and CTC architectures achieve real-time performance. Deployment must consider language-specific factors rather than assuming uniform performance across languages.

5.2 Failure Mode Analysis

5.2.1 LID Confusion and Error Cascading

Language identification errors directly cascade into transcription failures in LID→ASR pipelines. The overall LID accuracy of 93.25% falls below the 95% threshold generally recommended for production systems [13]. Mongolian exhibits the lowest accuracy (80%), while Spanish (97%), French (98%), and Hungarian (98%) achieve high accuracy. Mitigation strategies include confidence thresholding, using N-best LID hypotheses, and falling back to language-independent modes when uncertainty exceeds acceptable levels.

5.2.2 Low-Resource Language Degradation

Mongolian degradation across accuracy, speed, and consistency demonstrates low-resource language challenges in multilingual models. Technical analysis (through dialogue with the Chimege research group [14]) revealed specific causes: inference library optimization bias toward common languages, tokenization inefficiency (character-level for Mongolian vs. word-level for English), training data imbalance (most non-English languages have <1,000

hours in Whisper's corpus [5]), and architectural assumptions favoring high-resource patterns. The 40× slowdown for Whisper-small on GPU results from implementation optimization issues and tokenization strategy. Critically, larger Whisper models (medium, large-v3) achieve real-time Mongolian processing (RTF 0.55), indicating model capacity can compensate for these limitations.

5.2.3 Speed Variability and Predictability

Whisper-small exhibits high variability on Mongolian, with processing times varying significantly across samples. This unpredictability complicates deployment, as systems must account for worst-case latency and inconsistent response times can degrade user experience. OmniLingual models maintain low standard deviation relative to mean, providing predictable performance suitable for production.

5.3 Practical Deployment Recommendations

5.3.1 Model Selection Guidelines

Known High-Resource Languages (Spanish, English, French) Whisper-small provides reasonable performance (RTF 0.09–0.13 on GPU) with competitive accuracy. OmniLingual CTC models offer 30× speed advantage with comparable accuracy. OmniLingual LLM 1B balances speed and accuracy for quality-critical applications. Recommendation: Choose CTC models for speed-critical deployments, Whisper or LLM variants for accuracy-critical scenarios.

Medium-Resource Languages (Hungarian, Polish) Whisper-small achieves reasonable performance on Hungarian (RTF 0.13 on GPU). OmniLingual models remain highly capable (RTF 0.08–0.10). Recommendation: Both architectures suitable; CTC preferred for latency-critical applications.

Low-Resource Languages (Mongolian, minority languages) Whisper-small struggles (RTF 4.37 on GPU), but Whisper-medium/large-v3 achieve real-time performance (RTF 0.55). CTC models provide fastest option (RTF 0.08–0.10). Recommendation: Use larger Whisper models or CTC architectures; Whisper-small unsuitable.

Unknown Language Scenarios Requires LID→ASR pipeline. System reliability depends on LID accuracy. With LID accuracy >95%, automatic pipeline proves feasible. Current 93.25% overall accuracy (with Mongolian at 80%) suggests caution for production deployment without confidence thresholding.

5.3.2 Deployment Architecture Recommendations

Single Known Language Deploy one model instance conditioned on that language. Language-hinted mode eliminates LID overhead and errors. Prefer CTC models for fastest operation with acceptable accuracy. Trade-off: unified model simplicity vs. CTC speed.

Multiple Known Languages (including low-resource) Deploy OmniLingual CTC models or larger Whisper variants (medium, large-v3); Whisper-small is unsuitable due to low-resource language performance issues.

Unknown Languages Requiring Automatic Detection For high-resource languages, use Whisper with built-in LID (accepting 2× speed penalty). For low-resource languages, pair OmniLingual with external LID model, as Whisper performs poorly even with correct LID.

5.3.3 Hardware and Scaling

CPU Deployment (Edge Devices, Cost-sensitive) OmniLingual CTC models provide the most viable option. Whisper-small on CPU degrades significantly for Mongolian (RTF 10.58). Recommendation: CTC models preferred for CPU deployment; Whisper-small unsuitable for low-resource languages on CPU.

GPU Deployment (Server-side, Cloud) GPU acceleration significantly improves Whisper performance. Whisper-medium and large-v3 achieve real-time Mongolian processing (RTF 0.55), while Whisper-small remains impractical (RTF 4.37). OmniLingual maintains speed advantage (RTF 0.08–0.10) across all languages. Recommendation: Use larger Whisper models or CTC architectures for low-resource languages on GPU.

Mobile Deployment Extreme resource constraints favor lightweight models. OmniLingual CTC 300M most suitable. Recommendation: Deploy 300M CTC variant; larger models impractical for on-device inference.

5.4 Limitations

5.4.1 Dataset Limitations

Common Voice provides read speech with cleaner audio than spontaneous conversation [18]. Performance may degrade with background noise, overlapping speakers, and informal language. Audio samples predominantly fall in 0–10 second range, limiting long-form audio analysis. Only four languages evaluated, representing limited linguistic diversity.

A preliminary code-switching test concatenated audio from different languages (e.g., Spanish→Mongolian, French→Hungarian→Spanish) and tested Whisper's behavior. Results showed that Whisper performs LID once at utterance start and locks to that language throughout, failing to detect intra-utterance language switches. For example, Spanish-Mongolian concatenated audio was transcribed entirely in Spanish, ignoring the Mongolian segment.

A long-form drift test using concatenated French audio (120 seconds) revealed variable behavior: some samples maintained stable WER across time windows (avg 20-30%), while others showed significant drift with WER increasing to 80-98% in later segments. Language detection remained correct throughout, but transcription accuracy degraded in some cases, suggesting attention drift in extended audio. These preliminary findings indicate that comprehensive long-form and code-switching evaluation requires dedicated datasets and remains for future work.

5.4.2 Experimental Limitations

Initial Whisper-small experiments ran on CPU due to GPU compatibility issues. Subsequent GPU experiments with all Whisper variants (small, medium, large-v3) revealed the critical hardware-model interaction for low-resource languages. Using 1000 samples per language provides sufficient statistical power for estimating mean performance but limits analysis of rare phenomena. Greedy decoding employed for consistency. Beam search might improve accuracy at speed cost.

5.4.3 Metric Limitations

WER and CER prove sensitive to text normalization choices. RTF measures average speed but not latency distribution or concurrent load throughput. Automated metrics only partially reflect perceived quality, making perceptual evaluation a useful complement.

5.5 Comparison with Prior Work

5.5.1 Consistent with Prior Findings

Training data imbalance effects align with Pratap et al. (2020) [21] and Conneau et al. (2020) [22] observations on low-resource degradation. This evaluation extends findings beyond accuracy to significant speed penalties. Architecture effects on efficiency confirm Kim et al. (2017) [23] suggestions that non-autoregressive approaches offer lower latency, with magnitude varying dramatically by resource level (1.5–2× for high-resource, 40–50× for low-resource on GPU).

5.5.2 Novel Contributions

The 40× Mongolian degradation for Whisper-small and the finding that larger models eliminate this penalty represents important deployment guidance. CTC robustness indicates architectural principles can achieve fairer multilingual systems.

Chapter 6

Conclusions and Future Work

6.1 Answers to Research Questions

Using systematic evaluation, the thesis responded to five research questions:

RQ1 (LID Accuracy): LID accuracy varies by resource level. Spanish 97%, French 98%, Hungarian 98%, Mongolian 80%. Overall 93.25% falls below the 95% production threshold.

RQ2 (Inference Modes): Language-hinted mode eliminates LID error cascading. For low-resource languages with lower LID accuracy, explicit language hints are recommended.

RQ3 (Cross-Language Performance): Whisper-small shows 40× RTF variance between Mongolian and Spanish on GPU. Larger Whisper models reduce this to 2.5–3.4×. CTC models maintain constant RTF 0.08–0.10 across all languages.

RQ4 (Architecture): CTC processes 5–50× faster than encoder-decoder. Architecture dominates efficiency—similar parameter counts yield 49× speed difference on Mongolian.

RQ5 (Deployment): For low-resource real-time applications, use CTC models or larger Whisper variants. Whisper-small is unsuitable for low-resource languages regardless of hardware.

6.2 Limitations and Lessons Learned

Four languages represent limited linguistic diversity. Common Voice's read speech may not generalize to spontaneous conversation. Initial CPU-only Whisper experiments required subsequent GPU validation to reveal hardware-model interactions.

6.3 Future Work

6.3.1 Extended Evaluation and Model Optimization

Future work should expand to 10–15 languages spanning tonal (Mandarin), morphologically complex (Turkish, Finnish), and non-Latin scripts (Arabic, Hindi). Long-form audio and code-switching evaluation would reveal attention drift and language transition handling. Fine-tuning experiments could isolate whether Whisper's Mongolian slowdown reflects training data or architecture. Comparison with specialized models like Chimege would measure multilingual convenience versus accuracy.

6.3.2 Sign Language Accessibility

SignMate, a prototype speech-to-sign accessibility system, applies the architectural insights of this thesis to a three-step pipeline: ASR, text-to-gloss conversion, and gloss-to-avatar generation. CTC models' consistent latency makes them ideal for real-time sign generation. Future work includes end-to-end speech→sign models and sign intelligibility metrics.

6.4 Concluding Remarks

This thesis demonstrates that model architecture plays a central role in achieving equitable performance across languages. CTC-based models exhibit more consistent behavior across both high-resource and low-resource languages, while encoder–decoder models require careful choices in model capacity and hardware to remain viable for low-resource settings.

These findings suggest that robust multilingual ASR depends on architectural designs that maintain stable performance across resource levels, rather than prioritizing accuracy for high-resource languages alone.

Ultimately, the goal is automatic speech recognition systems that serve all language communities reliably.

References

1. G. Hinton, L. Deng, D. Yu, G. E. Dahl, A. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury, "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups," IEEE Signal Processing Magazine, vol. 29, no. 6, pp. 82–97, 2012. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6296526

2. Y. Liu, Q. Chen, Z. Li, H. Huang, M. Lu, and K. Yu, "Modular end-to-end automatic speech recognition framework for acoustic-to-word models," arXiv preprint, 2020. https://arxiv.org/pdf/2008.00953.pdf

3. A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," Advances in Neural Information Processing Systems (NeurIPS), 2020. https://arxiv.org/pdf/2006.11477.pdf

4. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," Advances in Neural Information Processing Systems (NeurIPS), 2017. https://arxiv.org/pdf/1706.03762.pdf

5. A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," OpenAI, 2022. https://cdn.openai.com/papers/whisper.pdf

6. A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," Proceedings of the 23rd International Conference on Machine Learning (ICML), 2006. https://www.cs.toronto.edu/~graves/icml_2006.pdf

7. A. Gandhe, H. Kuo, N. Kanda, Y. Gaur, Y. Gong, S. Watanabe, and T. Yoshioka, "ESB: A benchmark for multi-domain end-to-end speech recognition," arXiv preprint, 2022. https://arxiv.org/pdf/2210.13352.pdf

8. Omniglot, "Language and writing statistics," 2024. https://www.omniglot.com/writing/stats.htm

9. V. Pratap et al., "Scaling speech technology to 1,000+ languages," arXiv preprint, 2023. https://arxiv.org/pdf/2305.13516.pdf

10. Wikipedia, "List of languages by number of phonemes," 2025. https://en.wikipedia.org/wiki/List_of_languages_by_number_of_phonemes

11. X. Li et al., "Universal phone recognition with a multilingual allophone system," Proceedings of ICASSP, 2020. https://arxiv.org/pdf/2002.11800.pdf

12. Y. Peng, S. Dalmia, I. Lane, and S. Watanabe, "Branchformer: Parallel MLP-attention architectures to capture local and global context for speech recognition and understanding," Proceedings of the International Conference on Machine Learning (ICML), 2022. https://arxiv.org/pdf/2207.02971.pdf

13. W. Hou, Y. Dong, B. Zhuang, L. Yang, J. Shi, and T. Shinozaki, "Large-scale end-to-end multilingual speech recognition and language identification with multi-task learning," Proceedings of Interspeech, 2020. https://www.isca-archive.org/interspeech_2020/hou20_interspeech.pdf

14. Chimege Development Team, Personal communication regarding Mongolian ASR performance and implementation details, 2025. https://chimege.com/en/ Contact: adiyajav@chimege.com, bilguun@bolorsoft.com (Machine Learning Engineer at Bolorsoft).

15. Encyclopaedia Britannica, "Mongolian languages," 2025. https://www.britannica.com/topic/Mongolian-languages

16. Wikipedia, "Hungarian language," 2023. https://en.wikipedia.org/wiki/Hungarian_language

17. Instituto Cervantes, Spanish: A Language to the World 2025, Global Observatory of Spanish, 2025. https://observatoriodelespanol.cervantes.org/wp-content/uploads/2025/10/Spanish_a_language_to_the_world_2025.pdf

18. R. Ardila, M. Branson, K. Davis, M. Henretty, M. Kohler, J. Meyer, R. Morais, L. Saunders, F. M. Tyers, and G. Weber, "Common Voice: A massively multilingual speech corpus," Proceedings of LREC, 2020. https://arxiv.org/pdf/1912.06670.pdf

19. Meta AI, OmniLingual ASR: Open-source multilingual speech recognition system, GitHub repository, 2025. https://github.com/facebookresearch/omnilingual-asr

20. A. C. Morris, V. Maier, and P. Green, "From WER and RIL to MER and WIL: Improved evaluation measures for connected speech recognition," Proceedings of Interspeech, 2004. https://www.isca-archive.org/interspeech_2004/morris04_interspeech.pdf

21. V. Pratap, A. Sriram, P. Tomasello, A. Hannun, V. Liptchinsky, G. Synnaeve, and R. Collobert, "Massively multilingual ASR: 50 languages, 1 model, 1 billion parameters," arXiv preprint, 2020. https://arxiv.org/pdf/2007.03001.pdf

22. A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," arXiv preprint, 2020. https://arxiv.org/pdf/2006.13979.pdf

23. S. Kim, T. Hori, and S. Watanabe, "Joint CTC-attention based end-to-end speech recognition using multi-task learning," arXiv preprint, 2017. https://arxiv.org/pdf/1609.06773.pdf

Annex

I. Declaration on the Use of Generative Artificial Intelligence

☐ **I have not used** any generative AI tools.

X **I have used generative AI tools.** I have verified the content generated by AI, ensured the accuracy of the outputs, and properly indicated each instance of use in the table below.

Usage type	**Name of Generative AI Tool(s)**	**Affected Sections** (chapter, page number, reference)	**Estimated Proportion of Use** (per usage type)
Literature Review	Claude 3.5 Sonnet (Anthropic)	Chapter 2 (Background), Chapter 5 (Discussion - Comparison with Prior Work)	15%
Brief Summary of the Prompt	Find relevant academic papers for CTC speech recognition, Transformer architecture in ASR, and multilingual ASR evaluation. Provide citation information from these links.		
Program Code Generation	Claude 3.5 Sonnet (Anthropic, via Windsurf/Cascade IDE)	Data collection, plotting, WER/CER calculation scripts	70%
Brief Summary of the Prompt	Create Python scripts to: (1) collect RTF data from experiment results, (2) calculate WER and CER using jiwer library, (3) generate visualization plots for WER/CER comparisons, RTF analysis, confusion matrices, and speed-accuracy tradeoffs.		
Generating New Ideas or Solution Proposals	Claude 3.5 Sonnet (Anthropic)	Chapter 5 (Discussion -	10%

		Interpretation of Key Findings)	
Brief Summary of the Prompt	Analyze Whisper's 40x slowdown on Mongolian and suggest possible explanations related to architecture, training data, and tokenization.		
Creating an Outline (text structure, bullet points)	Claude 3.5 Sonnet (Anthropic)	Chapter 5 (Discussion - Interpretation of Key Findings)	20%
Brief Summary of the Prompt	Help me to structure the thesis. Suggest organization for results section.		
Creating Text Blocks	Claude 3.5 Sonnet (Anthropic)	All chapters (refinement and rephrasing)	30%
Brief Summary of the Prompt	Rephrase it for a more academic tone. Fix grammar issues. Check if chapter 1-6 is consistent with values and narrative?		
Generating Images for Illustrative Purposes			0%
Brief Summary of the Prompt			
Data Visualization, Generating Charts Based on Data Points	Claude 3.5 Sonnet (Anthropic)	Chapter 4 (Figures 4.1-4.18)	60%
Brief Summary of the Prompt	Generate Python plotting code using matplotlib and seaborn to create: WER/CER bar charts by model and language, RTF comparison plots, confusion matrix heatmaps, speed-accuracy scatter plots, duration distribution visualizations.		
Preparing a Presentation			N/A
Brief Summary of the Prompt			

Other (please specify)	LaTeX formatting and citation management with Claude 3.5 Sonnet (Anthropic)	References chapter, in-text citations throughout	40%
Brief Summary of the Prompt	I added a new citation as [30], renumber the citations. Adjust the formatting.		

Aggregated Percentage Value (for the core part of the task)	**25%**

Brief Textual Justification of the Aggregated Value:

The core intellectual contribution of this thesis (the research questions, experimental design, model selection, dataset curation, execution of ASR experiments on HPC infrastructure, interpretation of results, and formulation of conclusion) were performed bt the author.

Generative AI (Claude 3.5 Sonnet via Windsurf/Cascade IDE) was used as an assistive tool for:

1. Code generation (70% of script work): Phyton scripts for data collection, metric calculation, and visualization were generated with AI assistance. All scripts were. Reviewed, debugged, and validated by the author. The experimental data itself was collected through the author's own execution of ASR models on university HPC resources.
2. Writing refinement (30% of text): AI assisted with rephrasing paragraphs for clarity, fixing grammar, and ensuring consistency. All professional content, experimental findings, and analytical conclusions originated from the author.
3. Citation management (40% of reference work): AI helped locate appropriate academic references and format them correctly. All citations were verified by the author for accuracy and relevance.
4. Literature review support (15%): AI assisted in identifying relevant foundational papers. All papers were independently verified and read by the author.

The core scientific contributions (methodology, data collection, result interpretation, and findings about hardware-model interactions for low-resource ASR) represent the author's own work. AI served as an efficiency tool for mechanical tasks (coding, formatting, grammar) rather than a substitute for professional expertise.

All AI-generated content was verified for accuracy. The author takes full responsibility for the correctness of all content in this thesis.